

Vue 高频面试题

1、Vue 双向绑定原理

vue2.0 通过 `object.defineProperty` 劫持数据发生的改变，如果数据发生改变，就在 `set` 中进行赋值，触发 `update` 方法，进行更新节点内容`{{str}}`，从而实现了数据双向绑定

vue3.0 proxy 直接代理对象，不需要使用 `set`

2、Promise

1) 异步请求数据

在异步编程的基础上实现同步的结果。

实现承诺 承诺石沉大海 承诺等待结果

resolve reject pending

解决问题 拒绝考虑 苦苦等待

Promise 解决异步流程化的一种手段

promise 构造函数，需要 `new`

有且只有一个参数 `excutor` 执行器

`excutor` -> `resolve reject` 函数

`excutor` 在执行 `promise` 时调用

```
//(resolve,reject) =>{}  
//执行器，有两个参数resolve,reject  
//excutor是同步执行的  
let promise = new Promise((resolve, reject) => {  
  // resolve('承诺实现')  
  reject("承诺石沉大海");  
});  
//then是异步调用的  
promise.then(  
  (res) => {  
    console.log(res);  
  },  
  (err) => {  
    console.log(err);  
  }  
);  
console.log(11111);
```

pending->resolve

pending->reject

反过来是不行的，参数的位置：`resolve<->reject` 不能相互转换

2) async await

async 的意思是当前这个异步函数与同一作用域下的程序是异步的关系

await 是一个操作符

等待一个 Promise 对象产出结果的操作手段

功能是暂停 async 函数的执行，等待 Promise 处理后的结果

假如 Promise 处理结果是 rejected，会抛出异常

async 函数通过一个隐式的 Promise 返回 pending 状态

await 可以理解为隐式的 promise，await 后面的函数放入 promise 中执行，

await 语句后的代码放入.then 里面

```
await async();
console.log(11111);
console.log(22222);
////等价于
promise(() => {
  async();
}).then(() => {
  console.log(11111);
  console.log(22222);
});
```

基于生成器和协程实现

3) Promise.all

用于多个异步任务并发运行，它的结果创建承诺之后使用，等待所有任务结果的完成

可迭代 iterable 内部元素传递的是 Promise 对象集合，如果不是 promise，直接 resolve

iterable 内部没有元素，返回空数组

有一个 promise 是 rejected 实例回调 rejected

3、防抖

在一定时间(delay) 内多次触发时，只执行最后一次触发的回调函数

```
const inputDOM = document.getElementById("hInput");
// fn - 需要执行的回调函数
// delay - 防抖时间间隔(毫秒)
function debounce(fn, delay) {
  let timer = null;

  return function () {
    // 如果定时器已存在，则清除之前的定时器
    if (timer) {
      clearTimeout(timer);
    }
    // 设置新的定时器
    timer = setTimeout(() => {
      fn();
    }, delay);
  };
}
```

```

    }, delay);
  };
}

inputDOM.addEventListener(
  "input",
  debounce(() => {
    console.log("发送搜索请求");
  }, 500)
);

```

4、节流

一段时间内只触发、执行一次某个操作，过了这段时间，还有操作的话，继续执行新的操作。

```

const inputDOM = document.getElementById("hInput");

function jieliu(fn, delay) {
  let timer = null;

  return function () {
    // 如果定时器已存在,则不能在请求
    if (timer) {
      return;
    }
    timer = setTimeout(() => {
      fn();
      //执行完后清除定时器
      timer = null;
    }, delay);
  };
}

inputDOM.addEventListener(
  "input",
  jieliu(() => {
    console.log("发送搜索请求");
  }, 500)
);

```

防抖节流区别：

防抖：每次触发都会取消之前的延时调用

节流：每次触发事件时都会判断是否等待执行的延时函数

函数防抖连续触发的事件，只在最后一次执行

函数节流一段时间内执行一次

5、手写深拷贝

hasOwnProperty 判断属性是否为对象的自有属性，而不是继承来的属性(getter/setter 等)

浅拷贝：只复制对象的第一层属性，如果属性值是对象，则复制的是引用地址，改变时第一层不会改变原对象，第二层以上就会改变原对象（如果属性是基本类型，拷贝的是值本身，新旧对象互不影响。如果属性是引用类型，拷贝的是内存地址引用，新旧对象共享同一块堆内存数据。）例如：object.assign和扩展运算。

深拷贝：复制对象的所有属性，包括嵌套的对象属性，完全独立，改变不会影响原对象。例如：Lodash的 `cloneDeep()`。

```
//深拷贝的实现
function deepClone(obj = {}) {
  // 如果 obj 不是对象或数组，直接返回它
  if (typeof obj !== "object" || obj == null) {
    return obj;
  }

  let result;

  if (obj instanceof Array) {
    result = [];
  } else {
    result = {};
  }

  for (let key in obj) {
    if (obj.hasOwnProperty(key)) result[key] = deepClone(obj[key]); //递归拷贝子属性
  }

  return result;
}
```

6、渲染流程

1. 构建 DOM
2. 构建 CSSOM
3. 构建渲染树
4. 布局
5. 绘制

如果构建 DOM 的时候遇到了 JS，就请求下载执行 JS，JS 如果没有额外的设置，默认要等 CSSOM 构建完成。

而 JS 既可以对 DOM、也可以对 CSSOM 进行修改，这样后面的三步又会运行一次

7、从输入 url 到页面渲染完成的过程

1. 缓存检查

浏览器首先检查资源是否命中强缓存（`Cache-Control/Expires`）。若命中，直接使用本地缓存；否则发送请求，服务器可能返回 `304 Not Modified`（协商缓存生效），此时仍使用缓存。

2. DNS解析

将域名转换为IP地址，查询顺序为：浏览器缓存 → 系统（hosts）缓存 → 路由器缓存 → ISP DNS 服务器 → 递归查询（根域名服务器→顶级域名服务器→权威域名服务器）。

浏览器缓存：浏览器自身维护的DNS记录缓存，存储近期访问过的域名与IP的映射关系。

3. 建立连接与发送HTTP请求

- **TCP三次握手**：与服务器建立可靠连接。
- **TLS协商 (HTTPS)**：交换密钥、验证证书，建立加密通道。
- **发送HTTP请求**：携带请求头（如 `User-Agent`、`Cookie`）、方法（GET/POST等）、路径等。

4. 服务器响应与数据处理

服务器返回状态码（如 200 OK）、响应头（如 `Content-Type`）、响应体（HTML）。浏览器根据 `Content-Type` 决定处理方式（如 `text/html` 提交给渲染进程）。

5. 渲染进程处理与构建DOM树

- **解析HTML**：将字节流转换为DOM树，解析器遇到 `<script>` 会阻塞（除非使用 `async/defer`）。
- **预加载扫描器**：提前下载资源（如图片、CSS），优化加载效率。

6. 构建CSSOM树

解析CSS（内联/外部样式表），生成CSSOM树。CSS会阻塞渲染（避免FOUC），`@media` 允许按条件异步加载。

7. 生成渲染树 (Render Tree)

合并DOM与CSSOM，排除不可见节点（如 `display: none`），生成包含样式与结构的渲染树。

8. 布局 (Layout/Reflow)

计算每个节点的几何信息（位置、尺寸），受视口大小与 `<meta viewport>` 影响。布局变化（如窗口缩放）会触发回流。

9. 绘制与合成 (Paint & Composite)

- **绘制**：将渲染树转换为像素信息，分为多个图层。
- **合成**：由合成线程将图层分块栅格化，通过GPU加速合成最终界面。

优化关键点：

- **缓存策略**：合理设置 `Cache-Control` 减少请求。
- **减少重排/重绘**：使用 `transform` 替代 `top/left` 动画。
- **CDN加速**：缩短资源请求路径。
- **资源压缩**：如Gzip、图片格式优化。

8、虚拟 DOM

虚拟 DOM 本身是 js 对象

在代码渲染到页面之前，Vue 会把代码转换成一个对象(虚拟 DOM)。以对象的形式来描述真实的 DOM 结构，最终渲染到界面。

在每次数据发生变化前，虚拟 DOM 都会缓存一份，变化之时，现在的虚拟 DOM 会与缓存的虚拟 DOM 进行比较。在 VUE 内部封装了 diff 算法，通过这个算法来进行比较，渲染时修改改变的变化，没有发生改变通过原先的数据渲染。

9、DOM 的 diff 算法

虚拟 Dom 是真实 Dom 的 js 对象。

在每次数据发生变化前，虚拟 DOM 都会缓存一份，变化之时，现在的虚拟 DOM 会与缓存的虚拟 DOM 进行比较。在 Vue 内部封装了 diff 算法，通过这个算法来进行比较两个虚拟 DOM 之间的最小差异。

首先比较新老节点是否属于同类标签，如果不是同类标签则直接替换，如果是同类标签则比较新老节点是否相等，如果相等就直接返回，如果不相等则分情况讨论。

如果新老节点都有文本节点，则用新的文本替换旧文本

如果老节点没有子节点，新节点有子节点，则添加新的子节点

如果老节点有新的子节点，新节点没有子节点，则删除老的子节点

如果新老节点都有节点则执行 diff 的核心算法 updateChildren

所以**优化**diff 算法

- 1. 只比较同一层级，不跨层比较
- 2. 比较标签名，不同直接删除，不继续深度比较
- 3. 标签名和 Key 相同，就认为是相同节点，不继续深度比较

Vue 2 的 diff 算法：

双端比较： Vue 2 的 diff 算法采用双端比较策略，即同时从新旧节点的头部和尾部开始，进行四种情况的比较：新旧开始节点比较---新旧结束节点比较---旧开始节点与新结束节点比较---旧结束节点与新开始节点比较

依次比较，比较成功后退出当前比较，渲染结果以新节点为准，每次比较成功后 start 和 end 向中间靠拢，当新旧节点中有一个 start 跑到 end 之后**终止比较**

时间复杂度： 在最坏情况下，可能需要遍历新旧节点列表的每一个节点，时间复杂度为 $O(n^2)$ 。

Vue 3 的 diff 算法：

预处理相同前置和后置节点： 首先处理新日子节点中相同的前置和后置节点，减少需要比较的节点范围。

最长递增子序列(LIS)： 在处理剩余节点时，使用 LIS 算法找到新子节点中的最长递增子序列，以此确定哪些节点需要移动，哪些节点可以复用。

时间复杂度： 通过上述优化，Vue 3 的 diff 算法在大多数情况下将时间复杂度降低到 $O(n)$ 。

对比项	Vue 2	Vue 3 (优化)
Diff 触发	依赖 <code>Object.defineProperty</code> 监听	依赖 <code>Proxy</code> 监听
SameVnode 规则	key & 标签类型相同	key & 标签类型相同
Diff 方式	双端比较	双端比较 + 最长递增子序列
静态节点优化	❌ 不支持	✅ 静态提升（跳过静态节点）
Fragment 支持	❌ 需要根节点	✅ 允许多个根节点
性能优化	普通 Diff	跳过静态节点，优化列表更新

10、margin 负值

margin-top 负值 元素向上移动

margin-left 负值 元素向左移动

margin-bottom 负值 元素本身不变，下边元素向上移动

margin-right 负值 元素本身不变，右边元素向左移动

11、BFC (Block formatting context 块级格式化上下文)

1. css的定位方案

1. 普通流(默认)
2. 浮动：元素首先按照普通流的位置出现，然后根据 float 的方向，沿着水平方向移动，遇到障碍物才换行
3. 绝对定位：元素会整体脱离普通流，不占据空间，不会对其兄弟元素造成影响，也不会影响其他元素对其造成影响

2. BFC作用

BFC属于普通流，它创建一个独立的渲染区域，使得该区域内的元素不会影响外部元素，同时也不会被外部元素影响。简单来说，**BFC 内部的元素布局不会影响 BFC 外部的元素**，BFC 之间也相互独立。

- 清除浮动（防止父元素高度塌陷）
- 防止 margin 重叠
- 阻止浮动元素的重叠
- 确保多列布局不会相互干扰

3. 内部元素的渲染不会影响外界是什么意思？

BFC 内部元素的布局规则只会在这个 BFC 里面生效，不会影响 BFC 外部的元素。例如：

(1) BFC 可以防止 margin 合并

```
.container {
  width: 200px;
  background: lightblue;
}
.box1, .box2 {
  width: 100px;
  height: 50px;
  margin-bottom: 20px;
  background: orange;
}
<div class="container">
  <div class="box1"></div>
  <div class="box2"></div>
</div>
```

✗ 问题：margin 合并

- box1 和 box2 在普通流中，box1 的 margin-bottom: 20px 和 box2 的 margin-top: 20px 会合并成 20px，而不是 40px。

✓ 解决：让 .container 形成 BFC

```
.container {
  overflow: hidden; /* 触发 BFC */
}
```

- 由于 .container 形成了 BFC，它的子元素 box1 和 box2 之间的 margin 不会合并，而是会保留各自的 20px 间距，总共 40px。

(2) BFC 可以清除浮动

✗ 问题：父元素高度塌陷

```
.container {
  background: lightblue;
}
.float-box {
  width: 100px;
  height: 100px;
  background: orange;
  float: left;
}
<div class="container">
  <div class="float-box"></div>
</div>
```

- 由于 `float-box` 浮动，它不再占据普通流的位置，导致 `container` 没有内容撑开高度，背景色消失。

✓ 解决方案：给 `container` 触发 BFC

```
.container {
  overflow: hidden; /* 触发 BFC */
}
```

- 由于 `.container` 形成了 BFC，它可以包裹住内部的浮动元素，不会发生高度塌陷。

(3) BFC 使浮动元素不会重叠

```
<style>
.container {
  border: 2px solid black;
  padding: 10px;
}

.float-box {
  float: left;
  width: 100px;
  height: 100px;
  background: lightblue;
  margin-right: 10px;
}

/* 普通流中的元素（未触发 BFC） */
.normal-box {
  height: 150px;
  background: salmon;
}

/* 触发 BFC 的元素 */
.bfc-box {
  overflow: hidden; /* 触发 BFC */
  height: 150px;
  background: lightgreen;
```

```
}
</style>

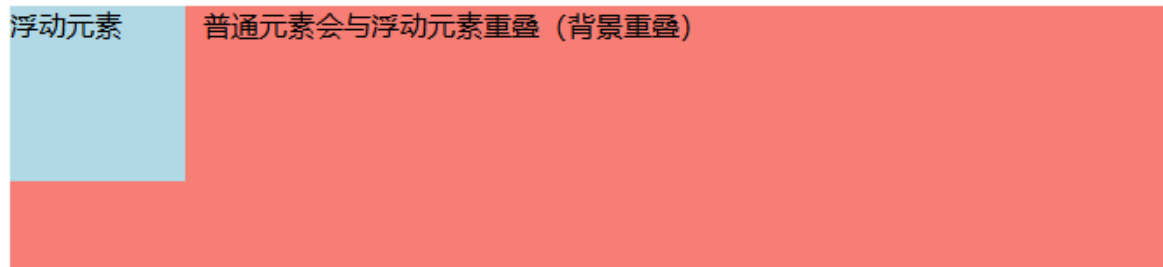
<div class="container">
  <!-- 未触发 BFC 的示例 -->
  <h3>未触发 BFC 的情况: </h3>
  <div class="float-box">浮动元素</div>
  <div class="normal-box">普通元素会与浮动元素重叠（背景重叠）</div>

  <hr>

  <!-- 触发 BFC 的示例 -->
  <h3>触发 BFC 的情况: </h3>
  <div class="float-box">浮动元素</div>
  <div class="bfc-box">BFC 元素不会与浮动元素重叠</div>
</div>
```

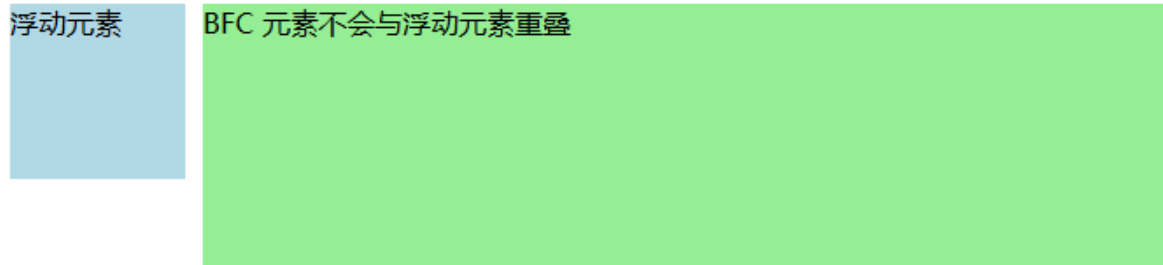
未触发 BFC 的情况:

浮动元素 普通元素会与浮动元素重叠（背景重叠）



触发 BFC 的情况:

浮动元素 BFC 元素不会与浮动元素重叠



4. 如何创建 BFC?

可以使用以下方式触发 BFC:

方式	代码
overflow 设为 hidden/auto/scroll	overflow: hidden;
display 设为 flex/grid/inline-block/table-cell	display: flex;
float 设为 left/right	float: left;
position 设为 absolute/fixed	position: absolute;

例如：

```
.container {  
  overflow: hidden; /* 触发 BFC */  
}
```

5. 总结

- BFC 是 CSS 视觉格式化模型的一部分，用于解决 margin 合并、浮动影响、布局隔离等问题。
- BFC 内部的元素不会影响外部，外部元素也不会影响 BFC 内部，形成“独立的渲染区域”。
- 触发 BFC 的方式包括 `overflow: hidden;`、`display: flex;`、`position: absolute;` 等。

在面试中，可以这样回答：

BFC（块级格式化上下文）是一种独立的渲染区域，它确保 BFC 内部的布局不会影响到外部，也不会受到外部的影响。它的作用包括防止 margin 合并、清除浮动、高度塌陷修复和避免浮动元素重叠。可以通过 `overflow: hidden;`、`display: flex;` 等方式触发 BFC。

12、圣杯布局

目的：

- 两侧内容宽度固定、中间内容自适应
- 三栏布局，中间一栏最先加载、渲染出来(主要内容)

实现方法：

1. float+margin
2. Flexbox 布局，通过 order 属性改变子元素的排列顺序

```
//order 实现  
.main {  
  display: flex;  
  flex: 1;  
  order: 2;  
  background: gray;  
  justify-content: center; //左右平移，居中，在一条x轴上  
  align-items: center; //上下平移，居中，在一条y轴上  
}  
.sidebar-left {  
  width: 200px;  
  order: 1;  
  background: pink;  
}  
.sidebar-right {  
  width: 200px;  
  order: 3;  
  background: yellow;  
}
```

13、flex

父级元素相关：

主轴：左右平移，水平居中;交叉轴：上下平移，垂直居中

1) flex-direction

主轴方向 水平方向(默认，起点在左端) row、row-reverse, 垂直方向 column、column-reverse

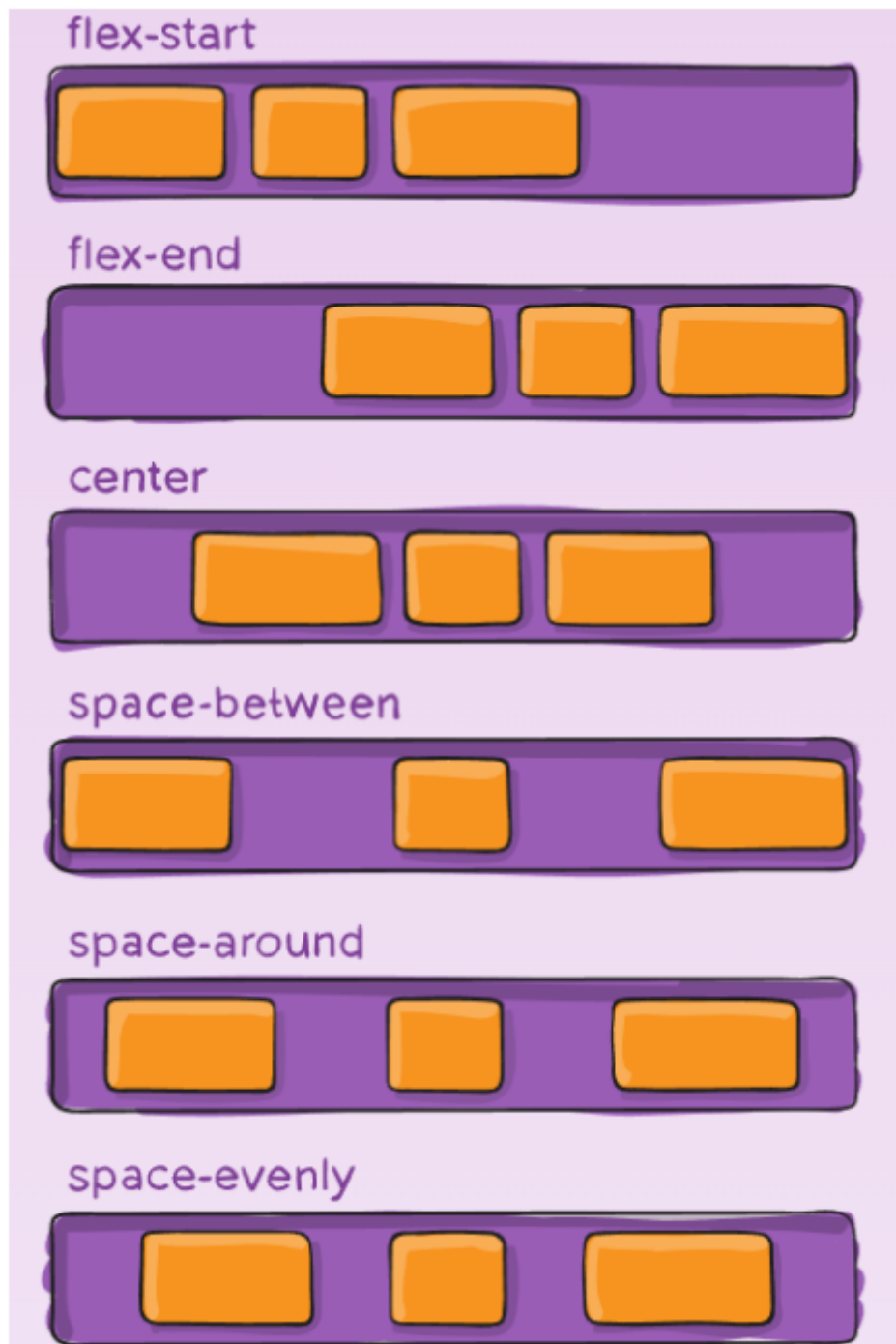


2) justify-content

主轴上的对齐方式 flex-start/flex-end/center/space-between/space-around

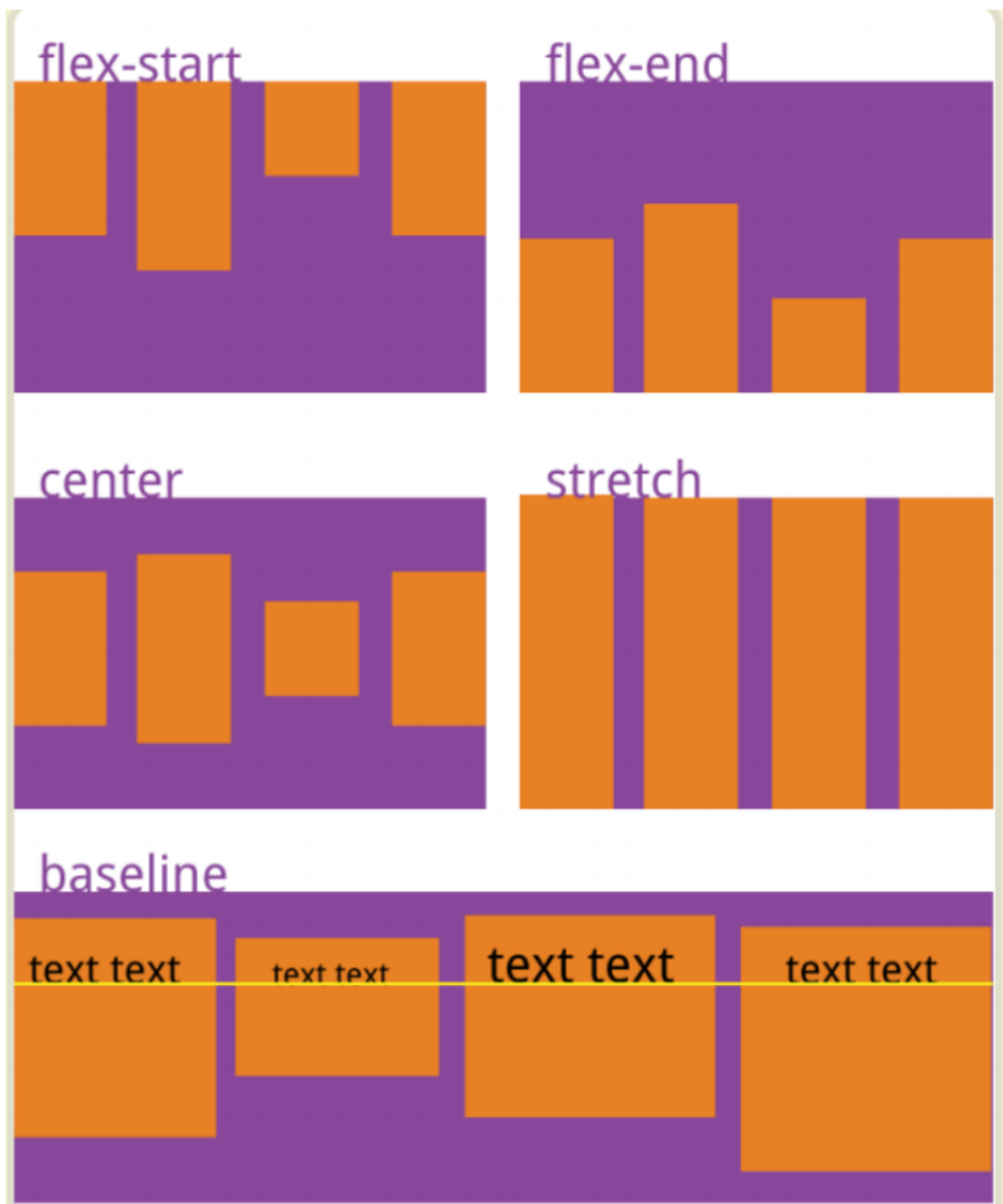
space-evenly 弹性项目之间的空白总是相等的

space-around 每个项目两侧之间间隔相等，所以项目之间间隔比项目与边框间隔大一倍



3) align-items

交叉轴上的对齐方式 flex-start/flex-end/center/baseline/stretch



4) flex-wrap

是否换行 nowrap/wrap/wrap-reverse

nowrap: 不换行

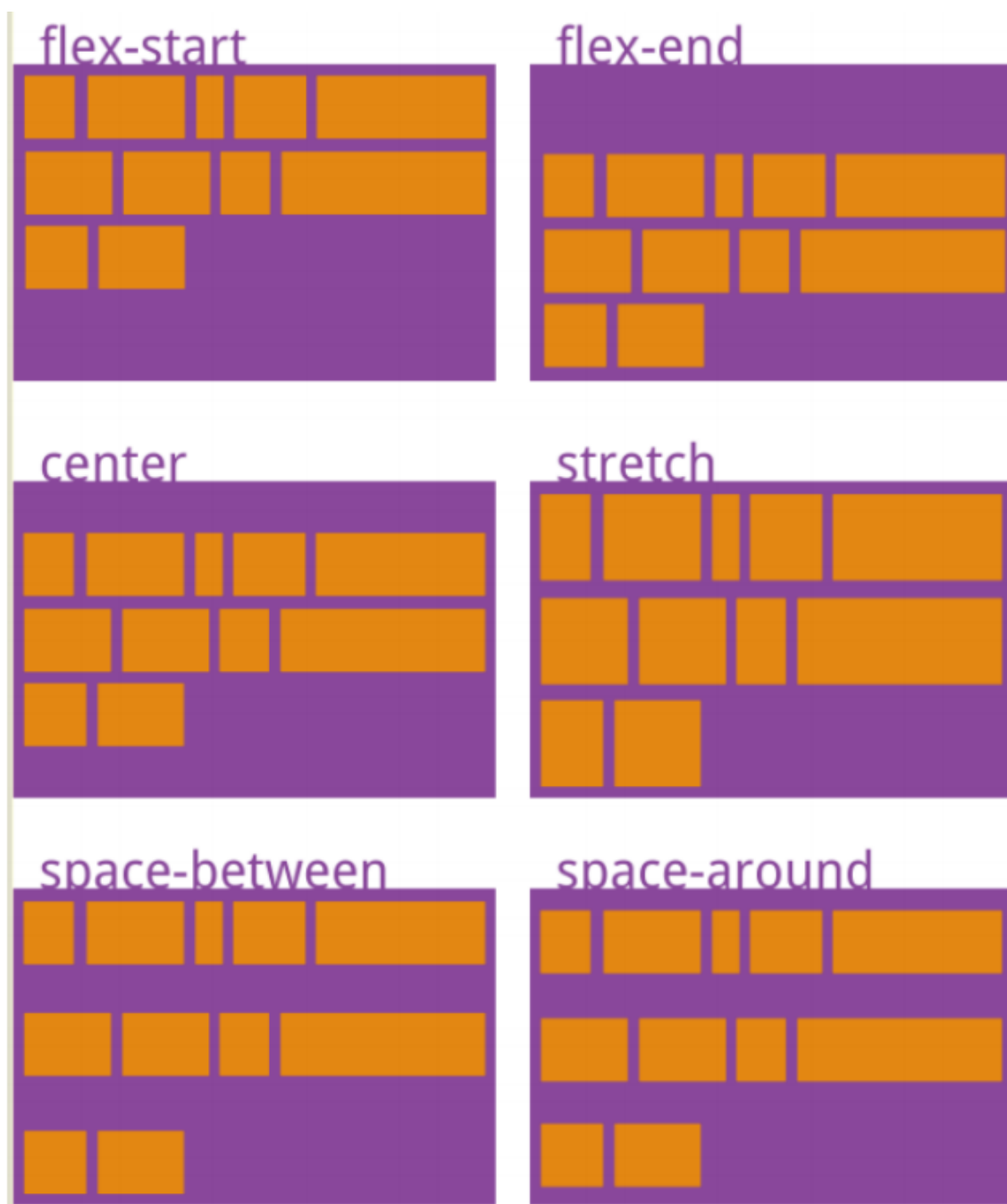
wrap: 换行, 第一行在上方

wrap-reverse: 换行, 第一行在下方

5) align-content

定义多根轴线的对齐方式, 如果项目只有一根轴线, 该属性不起作用。

flex-start/flex-end/center/space-between/space-around/stretch



子元素相关：

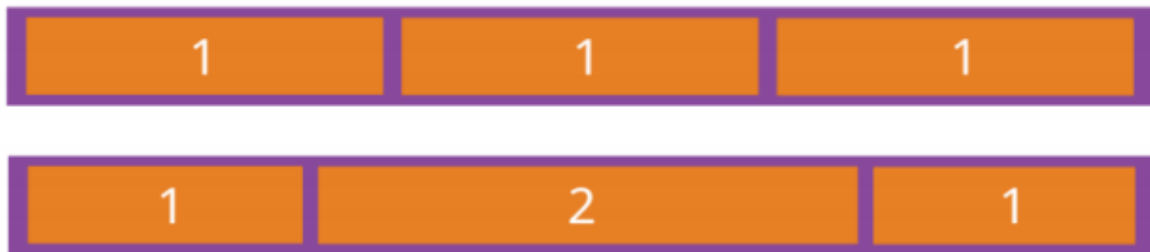
1) order

定义项目的排列顺序。数值越小，排列越靠前，默认为 0

2) flex-grow

定义项目的放大比例，默认值为 0，如果存在剩余空间也不放大

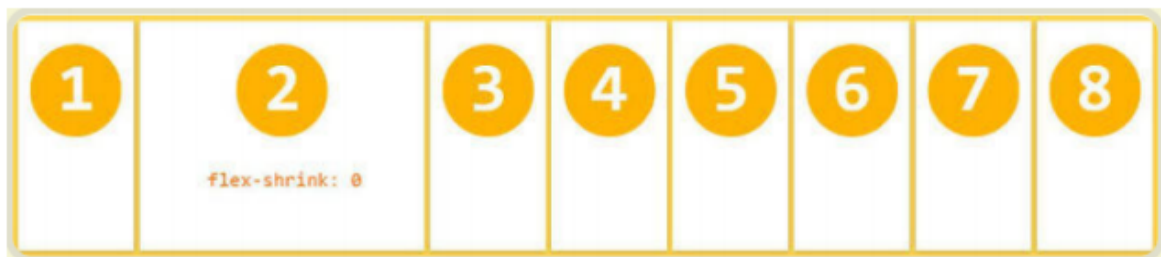
如果项目的 flex-grow 属性都为 1，则它们将等分剩余空间(如果有的话)。如果一个项目的 flex-grow 属性为 2，其他项目都为 1，则前者占据的空间将比其他项多一倍。



3) flex-shrink

定义了项目的缩小比例，默认为 1，即如果空间不足，则项目将缩小。

如果所有项目的 flex-shrink 属性都为 1，当空间不足时，都将等比例缩小。如果一个项目的 flex-shrink 属性为 0，其他项目都为 1，则空间不足时，前者不缩小。



4) flex-basis

定义了再分配多余空间之前，项目占据的主轴空间。浏览器根据这个属性，计算主轴是否有多余空间，默认值 auto，即项目本身的大小。

他可设置为与 width 或 height 属性一样的值，则项目占据固定空间。

5) flex

flex 属性是 flex-grow, flex-shrink 和 flex-basis 的简写，默认值为 0 1 auto，后两个属性可选。

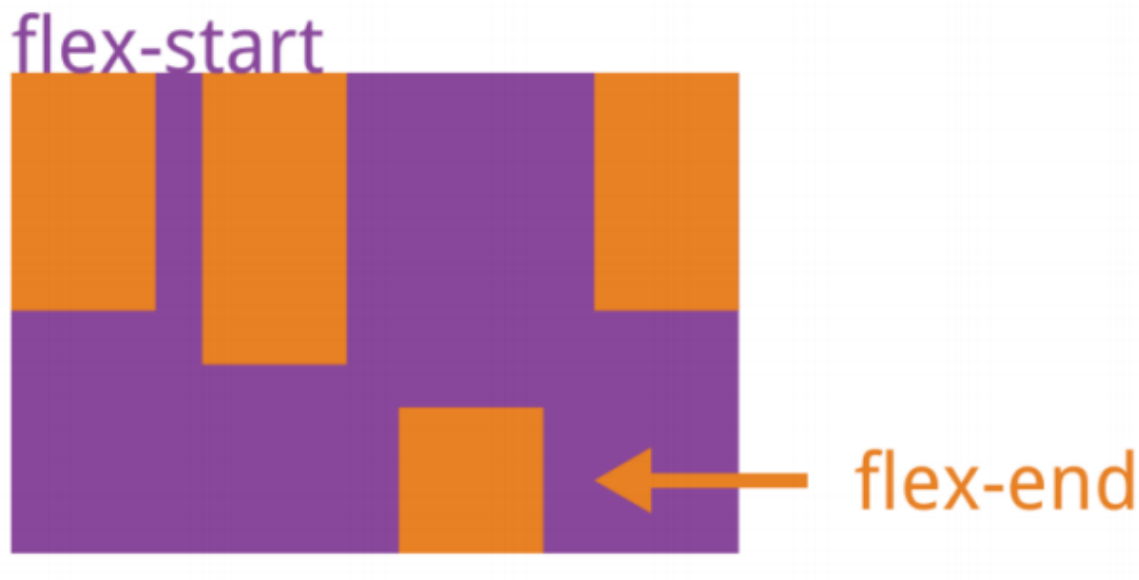
```
.item{
  flex: none | [<'flex-grow'><'flex-shrink'>? || <'flex-basis'>]
}
//该属性有两个快捷值: auto(1 1 auto)和 none(0 0 auto)
//建议优先使用这个属性，而不是单独写三个分离的属性，因为浏览器会推算相关值
```

6) align-self

子元素在交叉轴上的对齐方式，可以覆盖 align-items，默认值 auto 表示继承父元素的 align-items 属性，如果没有父元素，则等同于 stretch

align-self 允许单个项目与其他项目有不一样的对齐方式。

flex-start/flex-end/center/baseline/stretch/auto



14、relative、absolute

relative 定位： 相对于自身定位

absolute 定位： 相对于最近的一层父级定位元素(父级元素需要设置 position 属性为 relative、absolute、fixed)

定位元素position： relative、absolute、fixed

15、水平居中

1. 行内元素 inline 元素：text-align: center
2. 块级元素 block 元素：margin: 0 auto
3. absolute 定位元素：left:50%(父级元素的 50%) + margin-left 负值

16、水平垂直居中

1. 行内元素 inline 元素

line-height 的值==height 的值

2. 利用绝对定位，先将元素的左上角通过top:50%和left:50%定位到页面的中心，然后再通过margin负值来调整元素的中心点到页面的中心。该方法适用于盒子宽高已知的情況

```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  margin-top: -50px; /* 自身 height 的一半 */  
  margin-left: -50px; /* 自身 width 的一半 */  
}
```

3. 利用绝对定位，先将元素的左上角通过top:50%和left:50%定位到页面的中心，然后再通过translate来调整元素的中心点到页面的中心。该方法需要考虑浏览器兼容问题。

```
.parent {
  position: relative;
}

.child {
  position: absolute;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%); //让元素基于自身尺寸的 50% 进行水平和垂直方向的偏移
}
```

4. 利用绝对定位，设置四个方向的值都为0，并将margin设置为auto，由于宽高固定，因此对应方向实现平分，可以实现水平和垂直方向上的居中。该方法适用于盒子有宽高情况：

```
.parent {
  position: relative;
}

.child {
  position: absolute;
  top: 0;
  bottom: 0;
  left: 0;
  right: 0;
  margin: auto;
}
```

5. 使用flex布局，通过 align-items:center 和 justify-content:center 设置容器的垂直和水平方向上为居中对齐，然后它的子元素也可以实现垂直和水平的居中。该方法要**考虑兼容的问题**，该方法在移动端用的较多：

```
.parent {
  display: flex;
  justify-content:center;
  align-items:center;
}
```

17、什么是 rem

px：绝对长度单位

em：相对长度单位，相对于父元素

rem：相对长度单位，相对于 html 根元素

18、TCP 三次握手

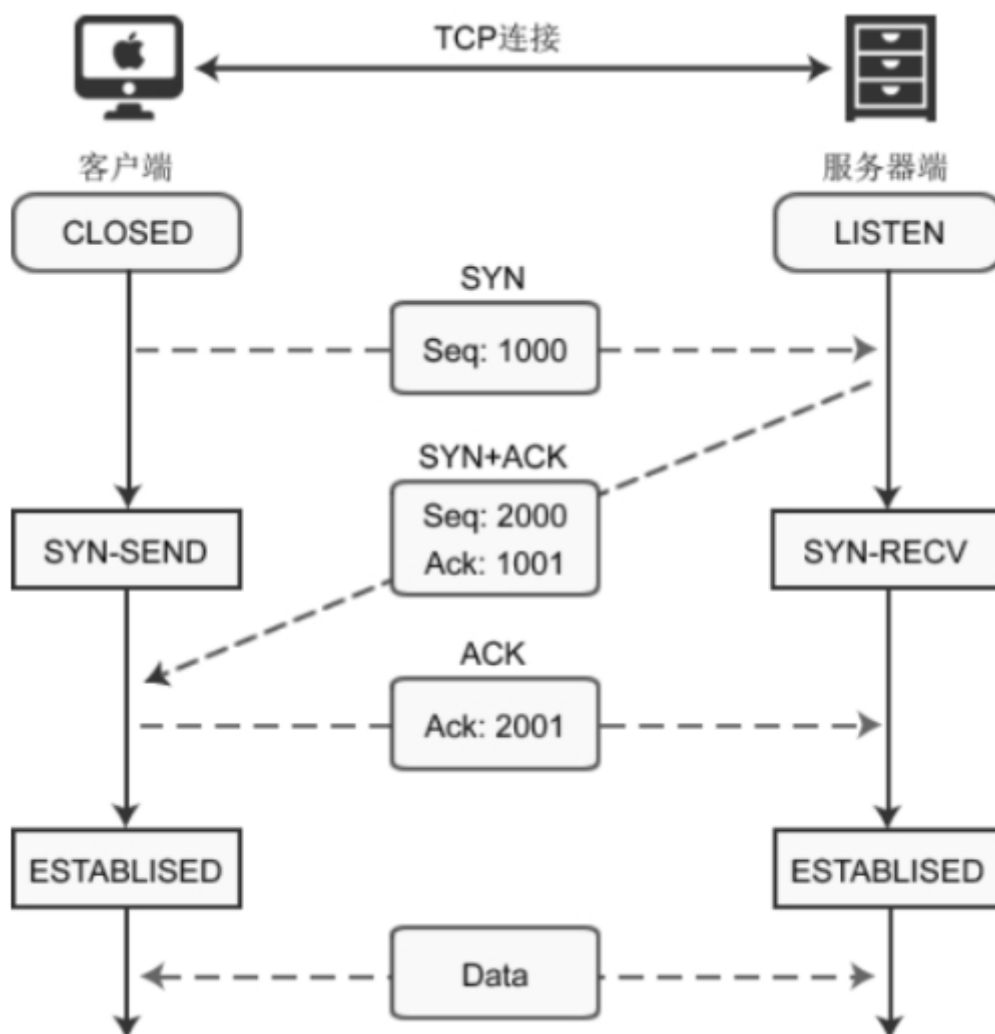
TCP 三次握手的主要目的是为了确认双方的接受能力和发送能力是否正常以及通过同步双方的序列号码和确认号码为后面的可靠性传输数据做准备，客户端向服务端申请 TCP 连接时，客户端执行 connect() 函数便会触发三次握手。

最开始，客户端处于 close 状态，服务端处于 listen 状态

第一次握手，客户端给服务端发送一个 SYN 报文，其中包含自己的初始化序列号码(seq=x)。此时客户端状态由 close 状态变为 SYN_send 状态

第二次握手，服务端收到客户端的 SYN 报文之后，会发送自己的 SYN 报文作为应答，并初始化自己的序列号码(seq=y) 以及确认号，其中确认号为客户端初始化序列号+1(ack=x+1)，此时服务端状态由 listen 变为 SYN_recv 状态

第三次握手，客户端收到服务器端的 SYN 报文后，会发送一个 ACK 报文，其中确认号为服务器端初始化序列号加一(ack=y+1)。此时客户端状态由 SYN_send 变为 established 状态。当服务器端收到客户端发来的 ACK 报文后，状态由 SYN_receive 变为 established 状态。连接建立成功。



2. 为什么不能只用两次握手?

如果使用两次握手（即客户端发送 SYN，服务器回复 SYN + ACK，就直接建立连接），可能会导致旧的重叠连接请求引发问题。

假设发生以下情况：

1. 客户端发送 SYN，但由于网络延迟，SYN 丢失，或者服务器未及时响应，客户端超时重传 SYN。
2. 服务器后来收到了旧的 SYN 请求，并建立了连接，但客户端可能已经放弃了这个请求（因为它可能已经重新发送了新的 SYN）。
3. 如果是两次握手，服务器在收到 SYN 就建立连接，可能会导致服务器资源浪费（僵尸连接），甚至引发错误的数据传输。

三次握手可以解决这个问题：

- **第三次握手 (ACK)** 确保客户端确认了服务器的 SYN，这样服务器才会正式进入 **ESTABLISHED** 状态，否则不会开启数据传输。
- 这样，即使旧的 SYN 重新到达，客户端不会再确认它，从而避免错误连接。

19、四次挥手

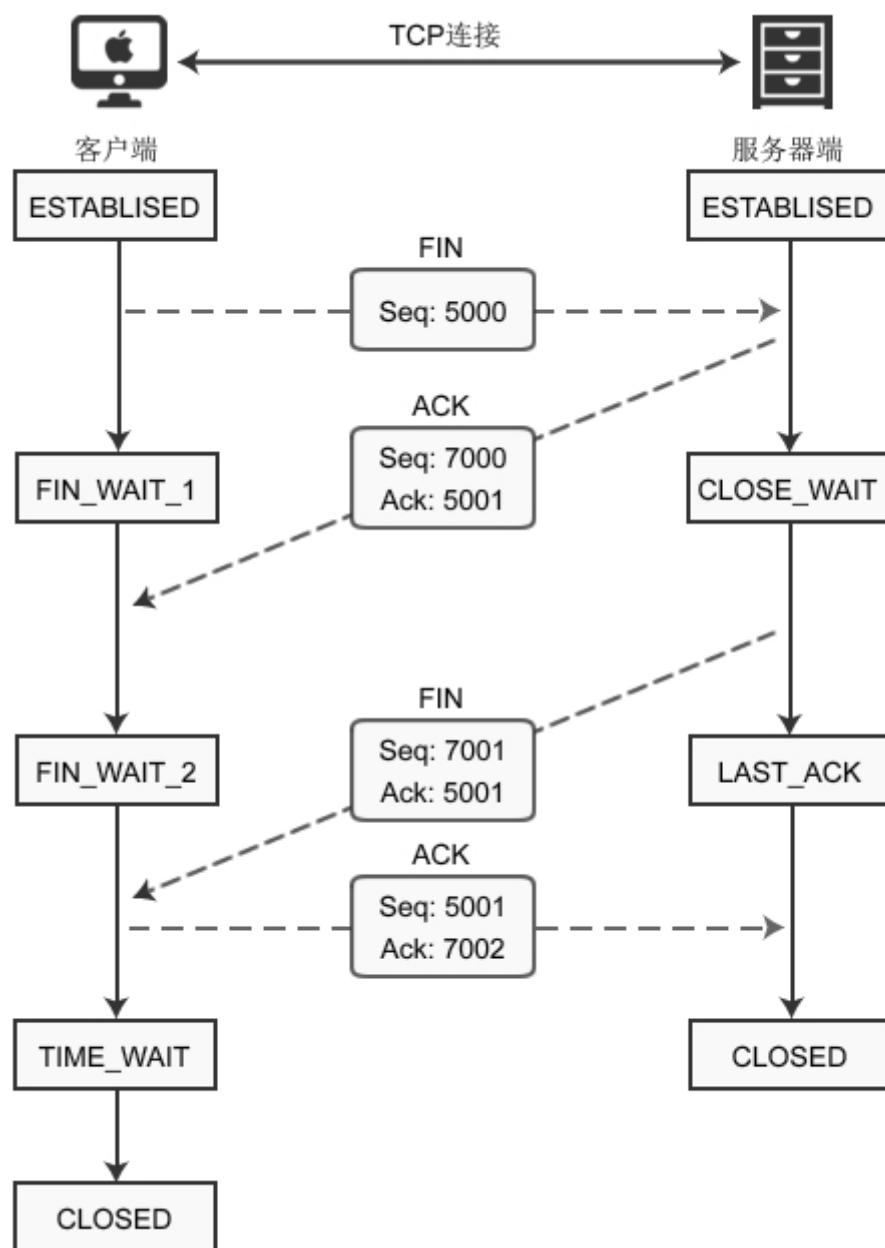
客户端向服务器端发起关闭请求。最开始双方都处于 established 状态

第一次挥手，客户端发送 FIN 报文和序列号 seq=x，并停止再发送数据，主动关闭 TCP 连接，进入 FIN_WAIT1 状态，等待服务端的确认。

第二次挥手，服务端收到 FIN 报文后，会发送确认报文 ACK，(确认号为 ack = x+1, seq=y)，此时服务端处于 CLOSE_WAIT 状态，此时 tcp 处于半关闭状态。客户端收到服务端的确认，进入 FIN_WAIT2 状态，等待服务端发出的连接释放报文。

第三次挥手，服务端没有要向客户发送的数据，服务端发送 FIN 报文(FIN=1, ACK=1, seq=w, ack=x+1)，服务端进入 LAST_ACK 状态，等待客户端的确认。

第四次挥手，客户端收到服务端的连接释放报文后，对此发出确认报文端(ACK=1, seq=x+1, ack = w+1) 客户端进入 TIME_WAIT 状态。此时 TCP 未释放掉，需要经过时间等待计时器设置的时间 2MSL 后，客户端才进入 CLOSED 状态。服务端在收到 ACK 报文后就进入 CLOSED 状态。



20、script 标签中 defer 和 async 的区别

defer 和 async 都用于外部脚本，但不同的是：

async 是异步加载，不确定什么时候加载好；页面加载时，带有 async 的脚本也加载，加载后会立即执行，如果有一些 DOM 的脚本加载慢时，这样会造成 DOM 没加载好，脚本就进行操作，会造成错误。无顺序加载。

defer：页面加载时，带有 defer 的脚本也同时加载，加载后会等待页面加载好后在执行。实现按序加载。

21、meta 标签

meta 主要用于字符编码、移动端适配、SEO 优化、重定向、缓存控制等。

核心必备 `<meta>` 标签：

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="网页描述，影响搜索引擎摘要">
```

适用于 SEO、移动端优化、网页控制等场景。

22、Vue3 与 vue2 的生命周期

vue3 的生命周期

setup()

onBeforeMount()

onMounted()

onBeforeUpdate()

BeforeUpdate()

onBeforeUnmount()

onUnmounted()

`<keep-alive>`

onActivated()

onDeactivated()

vue2 的生命周期

beforeCreate()

created()

beforeMount()

mounted()

beforeUpdate()

updated()

beforeDestroy()

destroyed

`<keep-alive>`

activated()

deactivated()

23、vue2 与 vue3 的区别

1. 双向绑定原理发生变化

vue2 利用 `Object.defineProperty()` 对数据进行劫持并结合发布者订阅者模式

vue3 使用 `proxy` 进行数据代理，可以实现对整个对象的监听，也可以检测到数组内部的变化

2. Vue3 的组件可以有多个根节点，Vue2 只能有一个根节点

3. vue2 是选项性 API，在代码里分割了不同的属性 `data`、`computed`、`methods` 等，Vue3 是组 合型 API 用方法来分割，这样代码会更整洁。

4. vue2 中响应数据直接放入 `data` 属性中，vue3 需要把响应数据放入 `setup()` 方法

5. 生命周期不同

6. vue3 的响应式系统使用 `weakmap`，当一个响应式对象已经被垃圾回收机制所回收时，就不需要 存储 `key-value` 了

24、vue3 与 react 区别

对比点	Vue3	React
编程思想	响应式编程，自动追踪依赖	函数式编程，手动管理状态
数据流	双向绑定， <code>v-model</code>	单向数据流， <code>props + state</code>
模板语法	HTML 模板 (<code>v-if</code> ， <code>v-for</code>)	JSX (<code>{}</code> 内可写 JavaScript 逻辑)
组件开发	Composition API (<code>setup</code>)	Hooks (<code>useState</code> ， <code>useEffect</code>)
状态管理	Vuex (推荐 Pinia)	Redux (可选 Recoil、Zustand)
性能优化	Block Tree Diff 算法，静态提升	Fiber 架构，需要手动 <code>useMemo</code>

25、Vue 双向绑定

vue2 采用数据劫持和发布者订阅者模式，通过 `Object.defineProperty()` 来劫持各个属性的 `setter` 和 `getter`，在数据发生变化时发布消息给订阅者，触发相应的监听回调。当数据发生变化时，Vue 会通知所有订阅者进行更新，在 Vue 中每一个挂载到视图上的组件、或者每一个 `watcher` 都可以看成一个订阅者，他们订阅了某一个数据的变化，并等待数据发生变化时进行更新。订阅者管理着数据和视图之间的关系，保证数据变化能及时反映到视图上。

vue3 基于 `Proxy` 的响应式系统更高效和灵活，自动支持动态属性，减少手动干预，结合 `track` 和 `trigger` 实现更精准的依赖追踪，显著提升性能和开发体验。

26、weakmap 与 map

`map` 的键可以是任何类型，`weakmap` 的键只能是引用类型（对象，数组，函数，正则（`RegExp`）和日期（`Date`））

`weakmap` 对 `key` 是弱引用，不影响垃圾回收机制工作，不会造成内存溢出

`weakmap` 使用场景，用于存储那些只有当 `key` 所引用的对象存在时，才有价值的信息

27、事件的捕获、冒泡机制

捕获：自顶向下

冒泡：自底向上

```
//监听的是什么阶段的事件
window.addEventListener

//冒泡阶段
window.addEventListener('click', () =>{})

//捕获阶段
window.addEventListener('click', () =>{}, true)
```

哪些场景用到这些机制？ **事件委托**

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
</ul>
```

```
const ul = document.querySelector("ul");
ul.addEventListener("click", function (e) {
  const target = e.target;
  if (target.tagName.toLowerCase() === "li") {
    const liList = this.querySelectorAll("li");
    const index = Array.prototype.indexOf.call(liList, target);
    //一个历史页面，上面有若干按钮的点击逻辑，每个按钮都有自己的 click 事件
    alert(`内容为${target.innerHTML}，索引为${index}`);
    //内容为2，索引为1
  }
});
```

给每一个访问的用户添加一个属性，banned=true，此用户点击页面上的任何按钮或者元素，都不可影响原来的函数。而是直接的 alert 提示，你被封禁了。

```
// 假设用户对象如下
const user = {
  banned: true, // 用户被封禁
  // 其他属性
};

// 在 document 上添加事件监听器，捕获阶段触发
document.addEventListener(
  "click",
  function (event) {
    if (user.banned) {
      // 阻止事件的进一步传播
      event.stopImmediatePropagation();
      // 提示用户被封禁
      alert("你被封禁了");
    }
  },
  true
); // 第三个参数为 true，表示在捕获阶段执行
```

28、promise

promise.all 有什么特性?

1. 并行执行与结果收集

输入: 接收一个可迭代对象(如数组), 其中每个元素为 Promise 实例或普通值。

行为: 立即并行执行所有 Promise, 并等待所有 Promise 完成。

结果: 返回一个新的 Promise, 其结果为所有 Promise 完成值的数组, 顺序与输入顺序一致。

2. 快速失败(Fast Fail) 机制

任一 Promise 被拒绝: 若输入的 Promise 中有一个被拒绝(reject), 则整个 Promise.all 立即拒绝, 返回第一个拒绝的原因。

3. 适用场景

多任务并行且依赖全部结果: 如同时请求多个接口数据, 需所有数据到位后渲染页面。

批量操作需原子性: 如多个文件上传, 任一失败则整体视为失败。

promise 的特性是什么?

1. 当创建 Promise 实例时, 其内部的 executor 函数(即 (resolve, reject) => {}) 会立即同步执行
2. 无法取消: 一旦 Promise 启动, 无法中途终止(需通过第三方库或封装逻辑实现取消功能)。
3. 状态不可逆: Promise 状态一旦变为 fulfilled 或 rejected, 不可再改变。
4. 异步结果处理: 即使 executor 同步调用 resolve, then/catch 中的回调仍会异步执行。

事件类型	执行顺序
同步代码	先执行 (当前任务栈中的代码)
Promise resolve/reject	调用 resolve() 或 reject() 后, 回调进入微任务队列
then/catch	then() / catch() 回调会在微任务队列中执行
宏任务	宏任务 (如 setTimeout) 在微任务队列清空后执行

29、Vue-router

hash 与 history 的使用

hash:

1. #后面就是 hash 的内容
2. 可以通过 location.hash 拿到
3. 可以通过 onhashchange 监听 hash 的改变

history:

1. history 即正常路径
2. 可以通过 location.pathname 拿到
3. 可以通过 onpopstate 监听 history 的变化

对比

调用 history.pushState() 相比于直接修改 hash, 存在以下优势:

- pushState() 设置的新 URL 可以是与当前 URL 同源的任意 URL; 而 hash 只可修改 # 后面的部分, 因此只能设置与当前 URL 同文档的 URL;
- pushState() 设置的新 URL 可以与当前 URL 一模一样, 这样也会把记录添加到栈中; 而 hash 设置的新值必须与原来不一样才会触发动作将记录添加到栈中;
- pushState() 通过 stateObject 参数可以添加任意类型的数据到记录中; 而 hash 只可添加短字符串;
- pushState() 可额外设置 title 属性供后续使用;

- hash 模式下，仅 hash 符号之前的 url 会被包含在请求中，后端如果没有做到对路由的全覆盖，也不会返回 404 错误；history 模式下，前端的 url 必须和实际向后端发起请求的 url 一致，如果没有对用的路由处理，将返回 404 错误。

30、服务端渲染和客户端渲染

1. 服务端渲染 SSR (Server-side rendering)

定义：在服务端完成页面内容的渲染，例如 JSP（是一种基于 Java 的 **服务端渲染** 技术）

特点：

1. 在服务端生成 html 网页的 dom 内容
2. 客户端只负责显示 dom 元素的内容

优点：可以被爬虫爬取，有利于搜索引擎优化 SEO(在服务器端生成完整的 HTML 页面，确保搜索引擎爬虫能够抓取和索引所有内容，从而提高 SEO 效果)

缺点：前后端不分离，开发效率低

使用场景：对搜索引擎优化要求较高的系统，比如门户首页、商品详情页

2. 客户端渲染 CSR (client-side rendering)

定义：向服务器发送 http 请求，获得想要的的数据，开始渲染 html 页面，生成 dom 元素，并将最终网页内容展示给用户

特点：

1. 服务端只给了客户端数据，而不是 html 页面
2. 客户端负责获取服务端的数据生成 dom 元素

优点：前后端分离，有利于提升开发效率

缺点：因为网站大量使用 javascript 技术，不利于爬虫爬取，不利于网站搜索引擎优化。这是因为在 CSR 模式下，网页内容主要通过 JavaScript 在客户端动态生成，某些搜索引擎的爬虫可能无法完全执行或索引这些动态内容，导致搜索引擎无法获取完整的页面信息。

使用场景：对搜索引擎优化 SEO 要求较低的系统，比如后台管理类系统

31、preload 和 prefetch

preload 是告诉浏览器页面必定加载的资源，浏览器一定加载这些资源，preload 将提升资源加载的优先级。

```
<link rel="preload" href="/img/all.jpg" as="image" />
```

prefetch 是告诉浏览器可能加载的资源，浏览器不一定加载这些资源

32、no-cache 和 no-store

都是 HTTP 协议头 Cache-Control 的值，区别是：

no-store：彻底禁用缓存。不会强制缓存也不会协商缓存，直接向服务器请求新资源。

no-cache：在浏览器使用缓存前，会往返对比 ETag，如果 Etag 没变返回 304，则使用缓存。使用 no-cache 的目的就是防止从缓存中获取过期资源。

33、jsonp

利用 `<script>` 标签天然支持跨域，可以在 `script` 标签的 `src` 属性中发送带有 `callback`(回调) 函数的 `get` 请求，服务端接收后将相应数据拼接到 `callback` 函数中，返回给浏览器，浏览器解析执行，从而前端能够拿到 `callback` 函数中响应数据。

实现步骤：

1. 在请求地址中拼接一个回调函数，得到一个新地址，将这个新地址赋值给 `<script>` 标签的 `src` 属性
2. 服务器接收这个回调函数，并向函数中注入参数，然后以字符串的形式返回这个函数以及其中的参数
3. `script` 标签在接受到返回的内容后，会将内容当做一段 `javascript` 代码自动执行

```
//Vue axios实现
this.$http = axios;
this.$http.jsonp('http://www.domain2.com:8080/login', {
  params: {},
  jsonp: 'handleCallback'
}).then((res) => {
  console.log(res);
})
```

```
//后端node.js代码:
var querystring = require('querystring');
var http = require('http');
var server = http.createServer();
server.on('request', function(req, res) {
  var params = querystring.parse(req.url.split('?')[1]);
  var fn = params.callback;
  // jsonp返回设置
  res.writeHead(200, { 'Content-Type': 'text/javascript' });
  res.write(fn + '(' + JSON.stringify(params) + ')');
  res.end();
});
server.listen('8080');
console.log('Server is running at port 8080...');
```

JSONP的缺点：

- 具有局限性，仅支持`get`方法
- 不安全，可能会遭受XSS攻击

34、伪数组

在JavaScript中，**伪数组（类数组）**是指具有 `length` 属性但不具有数组方法（如 `map`、`filter`）的对象，例如 `arguments`、`NodeList`、`HTMLCollection` 等。要将伪数组转换为真正的数组，可以使用以下几种方式：

1. `Array.from()`（推荐）

`Array.from()` 可以将**伪数组**和**可迭代对象**转换为数组。

```
const pseudoArray = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
const realArray = Array.from(pseudoArray);
console.log(realArray); // ['a', 'b', 'c']
```

适用于：

- `arguments`
 - `NodeList`
 - 具有 `length` 的对象
-

2. 扩展运算符 `[...]`

适用于可迭代对象，但不适用于普通的伪数组对象。

```
const nodeList = document.querySelectorAll('div');
const realArray = [...nodeList];
console.log(realArray);
```

适用于：

- `NodeList`
- `arguments` (可迭代对象)

⚠ 不适用于普通带 `length` 的对象，如 `{ 0: 'a', 1: 'b', length: 2 }`。

3. `Array.prototype.slice.call()`

`slice()` 可以将具有 `length` 属性的对象转换为数组。

```
const pseudoArray = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
const realArray = Array.prototype.slice.call(pseudoArray);
console.log(realArray); // ['a', 'b', 'c']
```

适用于：

- `arguments`
 - 具有 `length` 的对象
 - `NodeList`
-

4. `Array.prototype.concat.apply([], ...)`

利用 `concat()` 方法拼接空数组，将伪数组转为数组。

```
const pseudoArray = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
const realArray = [].concat.apply([], pseudoArray);
console.log(realArray); // ['a', 'b', 'c']
```

⚠ 性能不如 `Array.from()`，不推荐使用。

5. `for` 循环手动转换

```
const pseudoArray = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
const realArray = [];

for (let i = 0; i < pseudoArray.length; i++) {
  realArray.push(pseudoArray[i]);
}

console.log(realArray); // ['a', 'b', 'c']
```

适用于：

- 任何伪数组对象（性能稳定但代码较繁琐）

35、闭包

1. 闭包是什么？

闭包是指一个函数能够访问其外部作用域的变量，即使外部函数已经执行完毕。

2. 为什么要有闭包

1. 避免变量被污染
2. 私有化，外部只能使用不能修改
3. 保存变量，常驻内存

3. 应用场景：

1. 防抖节流
2. 库的封装(保证数据私有性)
3. 数据私有化（模拟私有变量）
4. 延迟执行（如 `setTimeout`）
5. 函数工厂（返回不同的函数实例）

```
function outer() {
  let count = 0; // 外部作用域变量
  return function inner() {
    count++; // 访问并修改外部作用域变量
    console.log(count);
  };
}

const counter = outer(); // 调用 outer，返回 inner
counter(); // 1
counter(); // 2
counter(); // 3
```

36、Object.create()

```
let obj = Object.create(null); //无原型链
let obj2 = {};
console.log(obj, obj2);
```

只是保存值，不用原型链上的方法用 `Object.create(null)`

37、白屏优化

白屏时间：页面出现第一个字符的时间-开始请求的时间

首屏时间：页面完成渲染的时间-开始请求的时间

DNS 查询：在页面中加入 dns-prefetch，在静态资源请求之前对域名进行解析，从而减少用户进入页面的等待时间。

```
<meta http-equiv="x-dns-prefetch-control" content="on" />
<link rel="dns-prefetch" href="https://s.google.com/" />
```

第一行中的 x-dns-prefetch-control 表示预解析功能。

第二行 dns-prefetch 表示强制对 s.google.com 做域名解析。这样在 s.google.com 资源请求开始前，DNS 解析完成，后续请求就不需要重复做解析了。

38、卡顿治理

1. 使用 transform 来代替直接设置 margin 等操作

transform 属性(如 translate、scale) 被浏览器标记为合成器属性，直接在独立的合成层进行变换，仅触发合成阶段，由浏览器的合成器线程直接处理，跳过布局(Layout) 和绘制(Paint) 阶段，性能更高。

2. 在对 DOM 元素增删的过程中最好先在 DocumentFragment 上操作，而不是直接在 DOM 上操作

	虚拟 DOM	真实 DOM	DocumentFragment
本质	JavaScript 对象	浏览器解析 HTML 生成的结构	真实 DOM 片段
是否渲染	不渲染	直接渲染	不能直接渲染
操作方式	先 diff 再更新	直接操作	批量更新后插入
适用场景	前端框架（如 React）	页面渲染	批量插入节点

3. 问题：复杂操作（如大量计算、同步 DOM 更新）如果一次性执行，会长时间占用主线程，阻塞页面渲染和用户交互，导致卡顿。

解决方案：通过**任务拆分**，将复杂操作分解为多个小任务，并利用 `setTimeout`、`requestAnimationFrame` 或微任务队列，在浏览器的空闲时间逐步执行，减少对主线程的占用，提高页面流畅度。

js统计页面加载完成需要的时间

```
//performance接口可以获取到当前页面中与性能相关的信息
//performance API 可以统计页面DOM渲染完成需要的时间
function getTiming(){
  setTimeout(()=>{
    let t= window.performance.timing;
    let performanceInfo =[
      {
        key: "Redirect",
        desc:"网页重定向的耗时",
        "value(ms)":t.redirectEnd -t.redirectstart
      },
      {
        key: "Appcache",
        desc:"检查本地缓存的耗时"
        "value(ms)":t.domainLookupstart -t.fetchstart
      },
    ],
```

```
.....]
  })
}
```

39、文档碎片

```
const list = document.getElementById("list");

const fragment = document.createDocumentFragment();

for (let i = 0; i < 5; i++) {
  const item = document.createElement("li");
  item.innerHTML = `项目${i}`;
  fragment.appendChild(item);
}

list.appendChild(fragment); //5次dom操作变1次
```

40、单页面与多页面

单页面应用(SPA)：

只在 web 页面初始化时，加载相应的 HTML、CSS、JS 等，一旦页面加载完成，就不会因为用户的操作而进行页面的重新加载和跳转，简单说就是 SPA 只有一个 web 页面。

通过监听路由的变化实现 HTML 内容的变换，从而动态实现 UI 与用户的交互，就像我们在 Vue 中从一个菜单项切换到另一个菜单项，页面内容虽然变了，但并没有做请求一个新 HTML 的动作，而通过变化的路由找到当前路由对应的页面，hash 路由就是 SPA 的应用。

优点：

1. 切换速度快、流畅、用户体验好
2. 前后端职责清晰，前端负责页面相关以及调后端接口拿数据的工作，后端负责数据处理

缺点：

1. 首屏加载慢(解决方法按需加载)
2. 不利于 SEO(搜索引擎优化)
动态内容生成：SPA 的内容由 JavaScript 动态渲染，传统爬虫(如早期 Googlebot) 可能无法执行 JS，导致无法索引动态内容。
无静态 HTML：初始 HTML 通常为空容器(如 `<div id="app"></div>`)，缺乏关键词和结构化数据，影响搜索引擎排名。

41、虚拟列表

虚拟列表就是对可见区域进行渲染，对非可见区域中的数据不渲染，以减少消耗，提高用户体验。是长列表的一种优化方案。

1. 写一个代表可视区域的 div 固定容器高度，通过 overflow 使其允许 y 轴纵向滚动
2. 计算可视区域内可以显示的数据条数，用可视区域高度/单条数据高度
3. 监听滚动，当滚动条变化的时候，计算出被卷起数据的高度
4. 计算可视区域内数据的起始索引=卷起数据高度/单条数据的高度
5. 计算可视区域内数据的结束索引=起始索引+可以显示的数据的条数
6. 取起始索引和结束索引中间的数据渲染到可视区域
7. 计算起始索引在整个列表中的偏移位置并设置到列表上

42、优化大体积图片

1. 通过 link 标签 preload 预加载

```
<link rel="preload" href="/img/all.jpg" as="image" />
```

2. 图片拆分
3. img 渲染完成后会触发 onload 事件，可以先隐藏图片，放一个图片或者 loading 进行过渡，然后图片触发 onload 事件后，进行切换
4. 把图片放到 cdn 上，可以通过 query 参数获得模糊图片，这样我们就可以实现模糊到清晰的渐进加载。

43、标签选择器优先级

! important > 行内样式 > ID 选择器 > 类选择器、伪类选择器、属性选择器 > 标签、伪元素 > 通配符 > 继承 > 浏览器默认属性

44、nextTick

1. 定义

`nextTick` 是 Vue 内部的异步队列调用方法，它允许开发者在数据变化并且 DOM 更新完成后执行某些操作，确保在后续逻辑中能够访问最新的 DOM 状态。

2. 执行时机

`nextTick` 在 DOM 异步渲染完成之后执行，通常是在数据更新、DOM 渲染完成后批量执行回调函数。

3. 使用场景

需要在数据变化后，执行依赖更新后的 DOM 的操作：**如果某些操作依赖于更新后的 DOM 结构，则应该将其放入 `nextTick()` 的回调函数中。**

在 Vue 生命周期钩子中，如果需要在 `created()` 钩子中操作 DOM，则应将 DOM 操作放在 `nextTick()` 的回调中，确保在挂载完成后执行。

`nextTick` 可以确保我们在 DOM 更新后执行需要依赖最新 DOM 状态的操作，特别是在 Vue 生命周期钩子和复杂的 DOM 操作中，它是一个非常有用的工具。

```
<template>
  <div>
    <p ref="text">Hello, vue!</p>
    <button @click="changeText">Change Text</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      message: 'Hello, vue!'
    };
  },
  methods: {
    changeText() {
      this.message = 'Text Changed!';
    }
  }
}
```

```
// 使用 nextTick 确保 DOM 更新完成后执行操作
this.$nextTick(() => {
  // DOM 更新完成后，获取最新的 DOM 元素
  const textElement = this.$refs.text;
  console.log('Updated Text:', textElement.textContent);
  // 可以在这里做一些 DOM 操作，如滚动到特定位置
});
}
}
};
</script>
```

45、打包优化

webpack-bundle-analyzer 分析打包结果

```
module.exports = {
  publicPath: './',
  configureWebpack: {
    externals: {
      'vue': 'Vue',
      'vue-router': 'VueRouter',
      'vuex': 'Vuex',
      'element-ui': 'ELEMENT',
      'axios': 'axios'
    }
  }
}
plugin: [
  new BundleAnalyzerPlugin()
]
```

不把 Vue、vuex、axios、Vue-router、element-ui 打包到文件中，减小打包体积，缩小首屏加载时间。然后使用 cdn 获取这些代码。

46、组合 api 和选项 api 的区别

选项 api：相当于添加对象的属性，vue2 中的 data，methods，computed

缺点：分散，功能逻辑不内聚

组合 api：基于函数

优点：在 setup() 中将用到的方法数据写出来，可以提到外面进行封装。

47、CommonJS 与 ES6 模块的区别

1. CommonJS 与 ES6 模块的比较

1. 浅拷贝与值引用：

- **CommonJS**：输出的是模块的浅拷贝，模块内部的修改不会同步到外部。
- **ES6**：输出的是值的引用，模块内部的修改会同步到外部。

2. 加载时机：

- **CommonJS**：在运行时加载模块。
- **ES6**：在编译时加载模块。

3. 导出与导入：

- **CommonJS**：使用 `module.exports` 导出，`require()` 导入。
- **ES6**：使用 `export` 导出，`import` 导入。

4. 导入提升：

- **CommonJS**：模块导入不会在编译期间提升。
- **ES6**：模块导入会在编译期间提升。

5. 顶层 `this`：

- **CommonJS**：顶层 `this` 指向模块本身。
- **ES6**：顶层 `this` 指向 `undefined`。

6. 加载方式：

- **CommonJS**：加载整个模块并引入所有接口。
- **ES6**：支持按需加载，可以单独加载某个接口。

7. 模块缓存：

- **CommonJS**：模块加载后会被缓存，`require()` 多次调用只会执行一次。
- **ES6**：模块加载后保持实时绑定，后续访问会反映最新的修改。

2. 模块化解决的问题

1. **加载顺序问题**：保证模块按正确顺序加载执行。
2. **污染全局变量**：通过模块化避免将变量暴露到全局作用域。

3. 模块化方案

1. CommonJS (适用于 Node.js)

- **模块引入**：使用 `require()` 引入模块，是同步方法，创建模块实例。
- **缓存机制**：多次 `require()` 只加载一次，避免重复加载。
- **局限性**：仅在服务器端 (Node.js) 运行，客户端不适用。

2. AMD (Asynchronous Module Definition, 异步模块定义)

- **适用场景**：前端，解决客户端的模块化问题。
- **模块引入**：使用 `require.js`，前置依赖管理，所有模块执行完毕后才执行回调函数。
- **解决的问题**：加载顺序问题，确保依赖模块加载完成后再执行。

3. CMD (Common Module Definition, 通用模块定义)

- **适用场景**：前端，由阿里推出，主要通过 `sea.js` 实现。
- **模块引入**：按需加载，依赖就近加载，灵活性更高。
- **AMD 与 CMD 的区别**：
 - **AMD**：依赖前置，模块加载时需要依赖全部加载完再执行。
 - **CMD**：依赖就近，按需加载，加载时仅加载当前模块需要的依赖。

4. 总结

特性	CommonJS	ES6
加载时机	运行时加载	编译时加载
输出类型	浅拷贝	值引用
导出/导入	<code>module.exports</code> / <code>require()</code>	<code>export</code> / <code>import</code>
顶层 <code>this</code>	指向模块本身	指向 <code>undefined</code>
按需加载	加载整个模块	按需加载
缓存机制	缓存模块，只执行一次	模块实时绑定，反映最新修改
适用场景	Node.js（服务器端）	前端开发（浏览器环境）

48、箭头函数与普通函数的区别

- 1、箭头函数的 `this` 在定义时就决定，而且不可修改(`call`, `apply`, `bind`)，箭头函数的 `this` 指向定义时外层第一个普通函数的 `this`。
- 2、箭头函数不能当做构造函数
- 3、箭头函数没有 `prototype`
- 4、箭头函数没有 `arguments`

特性	箭头函数	普通函数
语法	简洁，使用 <code>=></code>	使用 <code>function</code> 关键字
<code>this</code> 绑定	不绑定 <code>this</code> ，继承外部 <code>this</code>	根据调用方式绑定 <code>this</code>
<code>arguments</code>	无 <code>arguments</code> 对象	有 <code>arguments</code> 对象
作为构造函数	不能作为构造函数，不能用 <code>new</code> 调用	可以作为构造函数，用 <code>new</code> 创建实例
返回值	可以省略 <code>return</code> 和 <code>{}</code> ，简洁返回值	必须显式使用 <code>return</code> 返回值

49、axios

接口文档：

1. 请求路径
2. 请求方式
3. 请求参数
4. 响应数据示例

```

axios.get('url?参数1=值&参数2=值');
axios.post('url',{
  参数1: 值,
  参数2: 值
});
axios({
  url: '请求路径',

```

```

method: '请求方式，默认是get',
params: {}, //get请求方式，前端给后端传递的数据
data: {}, //post请求方式，前端给后端传递的数据
headers: {}, //自定义请求头
timeout: 1000, //请求超时，则请求中断
responseType: '默认是json格式的数据' //响应数据类型
})

```

```

import axios from "axios";
// import config from '../config'

// const baseUrl = process.env.NODE_ENV === 'development' ? config.baseUrl.dev :
config.baseUrl.pro
const baseUrl = "http: //172.27.86.114: 8080/jmetal_nudt/";
// 创建axios实例
const service = axios.create({
  baseUrl: baseUrl,
  timeout: 500000,
});

// 请求拦截器 请求发送到服务器之前对请求进行处理
service.interceptors.request.use(
  (configs) => {
    // 请求之前处理 判断用户是否为登录状态 添加token
    configs.headers.Authorization = window.sessionStorage.getItem("token");
    return configs;
  },
  (error) => {
    // 请求错误处理
    return Promise.reject(error);
  }
);

// 响应拦截器 后端给前端返回的数据
service.interceptors.response.use(
  (response) => {
    const res = response.data;
    if (res.status !== 0) {
      return Promise.reject(new Error("Error"));
    }
    return res;
  },
  (error) => {
    return Promise.reject(error);
  }
);

export default service;

```

50、cookie、localStorage、sessionStorage

特性	cookie	localStorage	sessionStorage
存储大小	约 4KB	约 10MB	约 5MB
存储时长	可设置过期时间	永久存储（除非手动删除）	仅在当前会话（Tab）有效，关闭浏览器后清除
作用域	服务器与前端均可访问	仅前端可访问	仅前端可访问
是否随请求发送	会（默认随 HTTP 请求发送到服务器）	不会	不会
适用场景	用于存储少量、需要与服务器通信的数据（如登录状态、认证、用户状态）	适用于长期存储大数据（如用户偏好设置）	适用于临时存储数据（如表单未提交数据）

1. Cookie 的安全性问题

✕ 存在的安全风险

1. 容易被 XSS（跨站脚本攻击）窃取

- 若页面存在 XSS 漏洞，攻击者可通过 `document.cookie` 获取 Cookie 数据。
- 防范措施：
 - 使用 `HttpOnly`：禁止 JavaScript 访问（仅服务器可读）。
 - 使用 `Secure`：仅 HTTPS 传输，防止中间人攻击。
 - CSP（内容安全策略）：限制外部脚本执行，防止 XSS。

2. 容易被 CSRF（跨站请求伪造）利用

- 由于 Cookie 会自动携带到服务器，攻击者可通过诱导用户访问恶意网站执行请求，借此篡改用户数据。
- 防范措施：
 - 使用 `SameSite=Strict` 或 `SameSite=Lax`：防止第三方网站携带 Cookie。
 - 使用 CSRF Token 进行请求验证。
 - Referer 头检查：确保请求来源可信。

3. 敏感信息泄露

- Cookie 存储的登录凭证、用户标识若被窃取，可能导致账户被劫持。
- 防范措施：
 - 避免存储敏感信息（如密码、Token），可以存储在服务器并使用 `sessionID` 代替。
 - Cookie 过期时间设置较短，防止长期有效。

2. localStorage 的安全性

✓ 优点

- 仅存储在浏览器，不会随请求发送给服务器，避免 CSRF。
- 存储容量比 Cookie 大（可达 5MB）。
- 适用于存储长期不变的数据，如用户设置、主题颜色等。

✗ 存在的安全风险

1. XSS 攻击（跨站脚本攻击）

- localStorage 可被 JavaScript 直接访问，若网站存在 XSS 漏洞，攻击者可读取 localStorage 并窃取 Token 或敏感信息。
- 防范措施：
 - 避免存储敏感信息，如用户 Token、密码。
 - 对输入进行严格的 XSS 过滤，防止恶意脚本注入。
 - 使用 CSP（内容安全策略），限制外部脚本执行。

2. 本地数据被篡改

- localStorage 可由用户或恶意脚本修改，若存储的是身份凭证，可能导致身份欺骗。
- 防范措施：
 - 避免将 localStorage 作为唯一身份验证手段，应结合服务器端验证。
 - 数据加密存储（如 AES）+ HMAC 校验完整性。

3. sessionStorage 的安全性

✓ 优点

- 仅存活于当前 Tab，不会跨窗口或标签页共享数据，相比 localStorage 更安全。
- 关闭浏览器 Tab 后数据自动清除，不会长期存储用户信息。

✗ 存在的安全风险

- 同样容易受到 XSS 攻击（sessionStorage 也可被 JavaScript 访问）。
- 数据不可跨标签页共享，若用户开多个页面，数据无法同步（可能是优点）。

防范措施：

- 避免存储敏感信息。
- 对输入进行 XSS 过滤，避免恶意代码执行。

4. 如何选择存储方式？

需求	建议使用	原因
需要在请求中携带数据（如 Session ID）	Cookie	服务器可自动读取
存储短期数据（如表单填写进度）	sessionStorage	关闭页面后自动清除，安全性更高
存储长期数据（如用户设置、Token）	localStorage	容量较大，但需注意 XSS 风险
存储敏感信息（如用户密码、Token）	✗ 不要存储在前端	推荐存储在服务器，使用 Token 机制

5. 进一步提升安全性

1. Cookie

- `HttpOnly` 防止 XSS 读取
- `Secure` 仅 HTTPS 传输
- `SameSite` 防止 CSRF
- 设置合理的过期时间

2. localStorage & sessionStorage

- 避免存储敏感数据（Token 应使用 HTTP `Authorization` 头）
- 防范 XSS（输入校验、CSP、安全编码）
- 数据加密（AES、Base64、HMAC 校验完整性）

总结

存储方式	是否安全	攻击风险	防范措施
Cookie	✗ 易受 CSRF & XSS 攻击	易被 XSS 读取、CSRF 劫持	<code>HttpOnly</code> 、 <code>Secure</code> 、 <code>SameSite</code>
localStorage	⚠ 易受 XSS 攻击	可被 XSS 窃取，数据可被篡改	避免存储敏感信息，CSP 防 XSS
sessionStorage	☑ 安全性较高	仍可被 XSS 窃取	避免存储敏感信息，CSP 防 XSS

推荐方案：

- 敏感数据不要存储在前端（Cookie、localStorage、sessionStorage）。
- 身份认证推荐使用 Token + HTTP Header（而不是 localStorage）。
- 防范 XSS、CSRF，增强前端安全策略（CSP、安全编码）。

🔒 安全第一，避免被攻击！

51、cookie、session、token

cookie 由服务器生成存储于客户端

session 存储于服务端

token 是服务端生成的一串字符串，相当于身份令牌。

52、服务器正常，前端白屏怎么定位错误

1. 检查浏览器控制台 (Console)

- 打开 开发者工具 (F12) → Console 面板
- 查看是否有 JavaScript 运行错误 (如 `SyntaxError`、`TypeError`)

2. 检查网络请求 (Network)

- 是否有 404、500 请求失败?
- API 请求是否返回正确数据?

3. 检查 `index.html` 是否正确加载

- 是否加载了 `index.html`?
- 是否成功引入 `JS`、`CSS` 文件? (Network 面板查看)

4. 检查是否有 JavaScript 代码异常

- 是否有 `console.log()` 输出?
- 是否有 `try-catch` 捕获错误?
- 是否引用了错误的变量或函数?

5. 检查是否有 `CSS` 样式导致白屏

- 是否 `body { display: none; }`?
- 尝试在 `body` 中插入 `div` 并检查是否可见

6. 关闭 `CDN` 或 缓存 试试

- 清除浏览器缓存 (`Ctrl + Shift + R` 强制刷新)
- 本地开发环境是否使用了 `CDN`? 是否需要替换为本地资源?

7. 启用 `Source Map` 还原错误

- 在 `webpack/vite` 项目中, 检查 `devtool: 'source-map'` 以便定位错误代码

总结

- ✓ 先看 `Console` 是否报错
- ✓ 再看 `Network` 是否加载失败
- ✓ 检查 `index.html` 和 `JS` 代码是否执行
- ✓ 尝试清除缓存、禁用 `CDN` 资源

53、Vue 登录

1. 登录流程

用户点击登录时, 会带着用户名和密码调用后端的登录接口

后端是验证数据库中有没有该数据, 如果没有则返回错误 (401 `unauthorized`, 该状态码表示认证失败, 用户提供的凭据 (密码) 无效), 前端会在页面提示用户, 如果信息存在, 则会给前端返回一个 `token`, 前端拿到 `token` 会存在 `vuex` 中并跳转到首页。

前端每次跳转到需要具备登录状态的页面是都需要判断 `token` 是否存在, 如果不存在则跳转到登录页面, 存在则正常跳转。通常把这个封装到路由守卫中。

再向后端发送其他请求时，需要在请求头上带上 token 信息，通常封装在请求拦截其中，后端判断请求头中有无该 token，有则正常返回数据，如果没有则返回相应的错误信息，前端拿到错误信息，清除 token，并且回退到登录页。

2. 将 token 存储到 cookie 中

vue-js-cookie 插件

```
npm install js-cookie
```

封装 auth.js

```
import Cookies from 'js-cookie'

const TokenKey = 'token'

//获取token
export function getToken() {
  return Cookies.get(TokenKey)
}

//设置token
export function setToken(token) {
  return Cookie.set(TokenKey, token)
}

//移除token
export function removeToken() {
  return Cookie.remove(TokenKey)
}
```

在 vuex 里配置 cookie

```
import Vue from "vue";
import Vuex from "vuex";
import { getToken, setToken, removeToken } from "../utils/auth";

Vue.use(Vuex);

export default new Vuex.Store({
  state: {
    token: getToken(),
  },
  mutations: {
    setToken(state, token) {
      setToken(token);
    },
    removeToken(state) {
      state.token = null;
      removeToken();
    },
  },
  actions: {},
});
```

3. 将 token 存储到 sessionStorage 中

```
window.sessionStorage.setItem("token", res.data.token);
```

请求拦截，给请求头加上 token

```
// 请求拦截器 请求发送到服务器之前对请求进行处理
service.interceptors.request.use(
  (configs) => {
    // 请求之前处理 判断用户是否为登录状态 添加token
    configs.headers.Authorization = window.sessionStorage.getItem("token");
    return configs;
  },
  (error) => {
    // 请求错误处理
    return Promise.reject(error);
  }
);
```

路由守卫

```
//为路由对象，添加beforeEach导航守卫
router.beforeEach(to, from, next) =>{
  //如果用户访问的是登录页，直接放行
  if(to.path===' /login') return next()
  const tokenStr = window.sessionStorage.getItem('token')
  //没有token，强制跳转到登录页
  if(!tokenStr) return next('/login')
  next()
}
```

路由守卫本质是 beforeEach 函数，该函数需要一个回调函数作为参数，回调函数中第一个参数 to，表示将要访问的地址，from 表示是从哪个地址中跳转过来的，next 表示继续执行的函数

JWT (JSON Web Token) 模式

JWT 是 Token 的一种具体实现方式，即基于 JSON 的令牌格式，使用 token 的认证模式，信息存储在客户端，服务端只负责校验。JWT 认证流程如下图：



54、项目优化

项目优化方法：

1. 生成打包报告
2. 第三方库启用 CDN
3. element-ui 组件按需加载
4. 路由懒加载
5. 首页内容定制

1) 进度条的使用

当数据加载慢时，可以展现出一个进度条

```
npm install --save nprogress
```

在 main.js 文件中导入 js 和 css 文件

```
//导入NProgress包对应的js和css
import NProgress from "nprogress";
import "nprogress/nprogress.css";
```

在发起请求时开启进度条，在获取响应时关闭进度条

```
//在请求拦截器中，展示进度条
axios.interceptors.request.use((config) => {
  NProgress.start();
  config.headers.Authorization = window.sessionStorage.getItem("token");
  return config;
});
//在响应拦截器中，隐藏进度条NProgress.done()
axios.interceptors.response.use((config) => {
  NProgress.done();
  return config;
});
```

2) 移除 console.log 指令

```
npm install bable-plugin-transform-remove-console --save-dev //安装
```

在项目根目录下面找到 babel.config.js 文件，做如下配置

```
//这是项目发布阶段需要用到的babel插件
const prodPlugins = [];
if (process.env.NODE_ENV === "production") {
  prodPlugins.push("transform-remove-console");
}
module.exports = {
  presets: ["@vue/cli-plugin-babel/preset"],
  plugins: [
    [
      "component",
      {
        libraryName: "element-ui",
        styleLibraryName: "theme-chalk",
      }
    ],
  ],
  //发布产品时的插件数组
```

```
...prodPlugins,  
]  
}
```

3) 生成打包报告

可以通过可视化的 ui 面板，方便看到项目中存在的问题

例如：依赖的文件(js、css) 比较大，页面打开速度慢

4) 打包优化

```
module.exports = {  
  publicPath: './',  
  configureWebpack: {  
    externals: {  
      'vue': 'Vue',  
      'vue-router': 'VueRouter',  
      'vuex': 'Vuex',  
      'element-ui': 'ELEMENT',  
      'axios': 'axios'  
    }  
  }  
  plugin: [  
    new BundleAnalyzerPlugin()  
  ]  
}
```

不把 Vue、Vuex、axios、Vue-router、element-ui 打包到文件中，减小打包体积，缩小首屏加载时间，使用 cdn 获取这些代码

5) 路由懒加载

可以实现路由被访问时才加载相应的组件

55、map 与 forEach 的区别

共同点：都可以拿来遍历数组，且每次执行匿名函数都支持三个参数，分别为 item(当前项)，index(当前索引值)，arr(原数组)。这个匿名函数中的 this 也都是指向 window。

区别：map() 有返回值，会分配内存空间，返回一个和原数组长度一致的新数组，基于这一点，map 可以进行链式操作，也就是说能够继续对新数组进行 filter，every 等等的操作；

forEach() 会改变原数组的值，没有返回值或者说返回的是一个 undefined，因此自然也就不能再链式操作。

应用场景：当我们调用后端接口返回的数据不满足我们的使用要求，我们想改变数组的数据时，就可以用 map，因为它会返回一个新数组，不影响原数组。

如果并不打算改变数据，只是想用数据做一些其他事，比如借助数组的值做一些和其他值无关的事情，就可以用 forEach。

```
//map  
const users = [  
  { id: 1, name: 'Alice' },  
  { id: 2, name: 'Bob' }  
];
```

```
// 提取ID集合
const userIds = users.map(user => user.id);
// 结果: [1, 2]

// 重组对象结构
const userProfiles = users.map(({ id, name }) => ({
  userId: id,
  fullName: name.toUpperCase()
}));
// 结果: [{userId:1, fullName:"ALICE"}, ...]
```

```
//foreach
// 多条件处理
const orders = [
  { id: 1, status: 'shipped' },
  { id: 2, status: 'pending' }
];

orders.forEach(order => {
  switch(order.status) {
    case 'shipped':
      sendConfirmationEmail(order.id);
      break;
    case 'pending':
      checkInventory(order.id);
      break;
  }
});
```

56、Vue computed 实现缓存原理

1.computed 是响应式的

computed 是响应式的，读取 computed 时就会触发 get，设置时会触发 set。

2.computed 是如何控制缓存的

某个计算属性 c 依赖 data 中的 a，如果没有缓存，每次读取 c 时，c 都会读取 a，就会触发 a 的 get，多次触发 a 的 get 是会消耗性能的，所以 computed 必须缓存。

脏数据标记: dirty，是 watcher 的属性

dirty 是 true 时，读取 computed 值时重新计算

dirty 是 false 时，读取 computed 值时会使用缓存

3.依赖的 data 发生改变，computed 是如何更新

收集依赖：计算属性 C 依赖于 data 中的 A，所以 A 收集计算属性 C 的 watcher。

数据变更，标记更新：当 A 发生变化，它会把 C 的 watcher 的 dirty 标记为 true(表示需要重新计算)。还会通知页面 P 的 watcher，使页面重新渲染。

计算属性重新计算：页面 P 重新渲染时，读取计算属性 C，发现 dirty 是 true，所以会重新计算 C。计算完成后，dirty 被重置为 false，并缓存结果。

后续读取：如果 A 没有再发生变化(dirty 是 false)，下次访问 C 时，直接返回缓存的值，不会重新计算，提升性能。

57、echarts 原理

1. 基于 ZRender 的图形抽象

底层渲染引擎：ECharts 依赖 ZRender 处理图形元素的绘制，ZRender 提供了基础的图形类(如圆形、矩形、路径等)和渲染能力。ECharts 在此基础上抽象出更高层次的图表元素(如折线、柱状图、饼图等)，简化了复杂图表的开发。

跨渲染器兼容：ZRender 封装了 Canvas 和 SVG 的底层差异，ECharts 可动态选择渲染方式（默认 Canvas），开发者无需关心具体实现细节。

2. 统一的事件机制

事件代理：ZRender 提供统一的事件系统(如点击、拖拽)，屏蔽了 Canvas 和 SVG 事件处理的差异。ECharts 通过监听这些事件实现图表交互(如数据项点击、图例切换)，并向上层暴露标准化的事件接口。

3. 单向数据流与函数式编程

单向数据流设计：自 ECharts 3.0 起，采用单向数据流模式。数据变化通过 setOption 更新配置，触发图表整体重绘。尽管描述为“全部重绘”，但内部通过差异比对(如配置合并、组件级更新)优化性能，并非无脑全量更新。

函数式思想：配置项(option)通常被视为不可变数据，每次更新生成新配置，避免副作用。这种设计易于状态管理，且与 React 等框架的不可变性理念契合。

4. 组件化架构

模块化组件：图表被拆解为独立组件(坐标轴、图例、系列等)，各组件根据数据变化自主决定渲染逻辑，减少不必要的更新。

分层渲染：ZRender 的图形树管理元素层级，ECharts 通过组件化控制渲染顺序，确保视图正确性(如提示框覆盖在图表上方)。

5. 性能优化策略

增量渲染：对于大数据量场景(如散点图、热力图)，ECharts 采用增量渲染和渐进式动画，避免卡顿。

Canvas 分层：动态内容(如动画)与静态内容分层渲染，减少重绘区域，提升性能。

58、post 与 get 的区别

1. 功能：

- **GET：**用于从服务器获取数据或资源。它是 **安全** 和 **幂等** 的，即多次请求相同的资源不会产生副作用。
- **POST：**用于向服务器发送数据，通常用于 **提交表单** 或 **上传文件**，可能会引起服务器状态的变化（例如创建或更新资源）。

2. 请求参数传递方式：

- **GET：**请求参数通过 **URL** 传递（例如：`https://example.com/api?name=John&age=30`）。参数会出现在浏览器地址栏。
- **POST：**请求参数通过 **请求体 (body)** 传递，不在 URL 中显示。参数通常是表单数据或 JSON 格式。

3. 数据长度限制：

- **GET**：由于 URL 长度限制（浏览器和服务器通常有不同的最大限制，通常在 2000 字符左右），因此传输的数据量有限。
- **POST**：没有固定的数据长度限制，理论上可以传输大量数据。

4. 缓存：

- **GET**：请求通常是 **可缓存的**，因为 GET 请求不修改服务器的状态，可以被缓存来提高效率。
- **POST**：请求 **不被缓存**，每次都会发送新的请求，因为 POST 请求可能会改变服务器状态。

5. 安全性：

- **GET**：由于数据通过 URL 传输，容易被 **窥探**（例如在浏览器的历史记录中），不适合传输敏感信息（如密码）。
- **POST**：由于数据通过请求体传输，相对更安全，特别适合传输敏感数据。

6. 幂等性：

定义：幂等性是指一个操作或请求无论执行多少次，结果都是相同的，不会对系统的状态产生不同的影响。

- **GET**：GET 请求是 **幂等的**，即多次发送相同请求不会对服务器的状态产生副作用。
- **POST**：POST 请求 **非幂等**，发送相同请求可能会导致不同的结果或状态变化（如重复提交表单）。

7. 使用场景：

- **GET**：用于 **数据查询** 或 **获取资源**，如搜索引擎请求、获取网页内容等。
- **POST**：用于 **提交数据** 或 **修改资源**，如用户登录、注册、表单提交等。

总结表格：

特性	GET	POST
功能	获取数据或资源	提交数据或修改资源
数据传输	数据通过 URL 传递	数据通过请求体传递
数据量限制	限制较大（URL 长度）	数据量较大，没有固定限制
缓存	可缓存	不缓存
安全性	数据暴露在 URL 中，不适合敏感信息	相对安全，数据不会暴露在 URL 中
幂等性	是（不会引起副作用）	否（可能导致状态变化）
常用场景	查询资源，搜索，浏览网页	提交表单，登录，上传文件

59、生成器（Generator）

生成器可以自由的切换线程，可以暂停和恢复一个函数的执行，使用 `yield` 关键字暂停函数执行，并通过 `next()` 方法恢复执行。

```
// 1. 定义生成器函数（带*号）
function* simpleGenerator() {
  yield '第一步';
  yield '第二步';
  return '完成';
}

// 2. 使用生成器
const gen = simpleGenerator();

console.log(gen.next().value); // "第一步"
console.log(gen.next().value); // "第二步"
console.log(gen.next().value); // undefined (已return)
```

60、fetch

```
fetch('url', {
  method: 'POST',
  body: JSON.stringify({ name:input.value }),
  headers: {
    'Content-Type':'application/json'
  }
})
.then(response => { response.json() })
.then(data => console.log(data) )
})
```

61、nginx

跨域配置: 在 `location` 配置 `proxy_pass`

```
server {
  listen      8089;
  server_name localhost;
  #charset koi8-r;
  #access_log logs/host.access.log main;
  location / {
    root   html;
    index index.html index.htm;
    try_files $uri $uri/ /index.html;
  }
  location /api/ {
    rewrite ^/api/(.*) $ /$1 break;
    proxy_pass http: //172.27.139.127: 8888/;
  }
}
```

监听到 8089 端口，以/api 开头，就反向代理到 `http: //172.27.139.127: 8888/`

配置 `rewrite ^/api/(.*) $ /$1 break;` 重写 url，把 api 干掉，后端就不用加 api 了

nginx 命令:

`nginx -t`: 检查配置文件是否有误

62、进程与线程

1) 概念:

进程是 CPU 资源分配的最小单位，线程是 CPU 调度的最小单位

进程是一个程序运行实例，当开启一个程序时，操作系统会为该程序分配一块内存，用来存放代码、运行中的数据、和执行任务的主线程，这样一个运行环境称为进程。

进程与线程之间的关系:

1. 一个线程崩溃会导致整个进程崩溃
2. 当进程关闭，操作系统会回收该进程的所有资源
3. 进程之间内容隔离
4. 一个进程可以有多个线程，多个线程共享进程资源

进程:

1. 浏览器主进程
2. GPU 进程
3. 网络进程
4. 渲染进程
5. 插件进程

渲染进程中的线程:

1. GUI 线程
2. js 引擎线程
3. 事件触发线程
4. 定时器线程
5. 异步 HTTP 请求线程

2) 进程与线程区别:

1. 进程是 cpu 资源分配的最小单位，线程是 cpu 调度的最小单位，一个进程可以有多个线程
2. 线程共享同一进程中的资源，进程间的通信需要借助进程间的通信
3. 进程间的切换开销比线程间的切换开销大

3) 进程间的通信方式

1. 管道通信
2. 消息队列
3. 信号量
4. 共享内存通信
5. socket 通信

4) 死锁产生的原因:

1. 进程竞争不可剥夺资源或者临时资源
2. 进程间推进顺序非法

死锁的必要条件:

1. 互斥条件
2. 请求和保持条件

3. 不剥夺条件
4. 环路等待条件

如何解决死锁：

1. 破坏请求条件：一次性分配所有资源，这样就不会再请求资源
2. 破坏请求和保持资源：一个进程分配不到资源，也不给他分配其他资源
3. 破坏不剥夺条件：一个进程已有部分资源，再请求资源导致阻塞，就释放他现有的资源
4. 破坏环路等待条件：给所有资源进行编号，进程按递增顺序请求资源，按递减顺序释放资源。

63、多个标签页通信

1) localStorage(推荐)

标签页对 localStorage 变化事件进行监听，需要标签页同源(不跨域)才能使用

2) websocket

通过 WebSocket 服务器作为消息中转站，所有标签页连接到同一个 WebSocket 服务。当一个标签页发送消息时，服务器将消息广播给其他所有连接的客户端标签页，从而实现多标签页通信。

1. 保持连接状态，而 Http 是无状态连接即请求完毕后会关闭连接
2. 全双工通信，客户端和服务端平等对待，可以相互通信
3. 建立在 TCP 协议上
4. 可以实现跨域

缺点：需要新建服务器，浪费资源，后期需要维护

3) shareworker 是 webworker 的一种，可以由所有同源页面共享

shareWorker 会在页面存在的生命周期内创建一个唯一的线程，并且开启多个页面也只会使用同一个线程。这个时候共享线程就可以充当中介者的角色。标签页间通过共享一个线程，然后通过这个共享的线程来实现数据的交换。

缺点：多个标签页不能跨域类似于 localStorage，调试不方便，兼容性不好

64、为什么 js 是单线程的语言

因为最初的 js 是用来在浏览器验证表单操纵 DOM 元素的。如果 js 是多线程的话，两个线程同时对一个 DOM 进行了相互冲突的操作，那么浏览器的解析是无法执行的。

webworker 的作用是为 js 创建多线程环境，允许主线程创建 worker 线程，将一些任务分配给后台运行。主线程运行的同时，worker 线程在后台运行，两者互不干扰。等到 worker 线程完成计算任务，在把结果返回给主线程。

65、css 阻塞

css 不会阻塞 dom 树解析

css 阻塞 dom 树渲染

css 会阻塞 js 执行

66、伪元素与伪类

在 css3 中伪类用 `:` 表示，伪元素用 `::` 表示

用于选择元素在 **特定状态或位置** 时的样式

```
/* 已选中的复选框 */
input: checked {
  background: green;
}
/* 第一个子元素 */
li: first-child {
  font-weight: bold;
}
```

伪元素创建了一些不在 DOM 树中的元素，并为其添加样式，必须有 content 属性，用于向元素的 **特定部分插入虚拟内容** 或样式。

语法：以双冒号 :: 开头（CSS3 规范），如 ::before、::after。

```
/* 在元素后添加装饰线 */
.title::after {
  content: "";
  display: block;
  width: 50px;
  height: 2px;
  background: black;
  margin-top: 5px;
}
/* 修改文本选中颜色 */
::selection {
  background: yellow;
  color: black;
}
```

核心区别是否创建了新的元素：伪类的操作对象是 DOM 中已有的元素，伪元素是创建 DOM 树以外的元素并为其添加样式

67、transform

1. 盒模型尺寸和位置不会发生变化

transform 的变换发生在「**合成层**」阶段（浏览器渲染流程的最后一步），不影响之前的布局计算（不会造成重排和重绘）。

2. 创建层叠上下文

任何 transform 作用的元素都会 **自动创建一个新的层叠上下文**，类似 z-index 触发的效果，其子元素的 z-index **不会影响** 外部的兄弟元素。

3. 内联元素无效

transform 对**内联元素**（如 、<a>）默认无效，需将 inline 改为 inline-block 或 block。

CSS 规范限制：内联元素的渲染模型不直接支持坐标系变换。

特别的，替换内联元素（如 、<svg>）可应用 transform。

4. 不同顺序不同效果

变换函数的顺序不同会导致最终效果不同，因为变换矩阵的乘法顺序不可交换。

5. 锯齿或虚化问题

锯齿 (Jagged Edges)：通常出现在 **旋转、倾斜** 时，因为像素对齐不规则，导致边缘不光滑。

虚化 (Blur)：通常出现在 **缩放** 时，浏览器可能进行 **插值计算**，导致模糊。

```
//优化
div {
  transform: scale(1.5);
  image-rendering: crisp-edges; /* 针对图像 */
  backface-visibility: hidden; /* 减少 3D 变换带来的模糊 */
  will-change: transform; /* 提前让浏览器优化 */
}
```

68、替换元素

加载的是外部内容，并不受现有的 CSS 属性影响，通过 CSS 进行设置

```
<iframe/>
<img/>
<video/>
```

69、层叠上下文

触发层叠上下文的条件：

`position: absolute|relative`，`z-index` 为数值

1. 层叠上下文可以包含在其他层叠上下文中，并且组建了一个有层级的层叠上下文
2. 每个层叠上下文完全独立与它的兄弟元素，处理层叠时只考虑子元素，这里类似于 BFC（块级格式化上下文）



70、清除浮动

解决父容器高度塌陷

1. 将浮动元素的父级标签设置为 BFC，`overflow: hidden`
2. 最后一个浮动元素后，添加一个空的标签，设置 `clear: both`

71、原型与原型链

原型：函数都有 prototype 属性，称之为原型，也叫原型对象

- 原型可以放一些属性和方法，共享给实例对象使用
- 原型可以做继承

原型链：对象都有 **proto** 属性，这个属性指向它的原型对象，原型对象也是对象也会有 proto 属性，指向原型对象的 **原型对象**，这样一层一层形成的链式结构称为原型链，不断向上查找直到

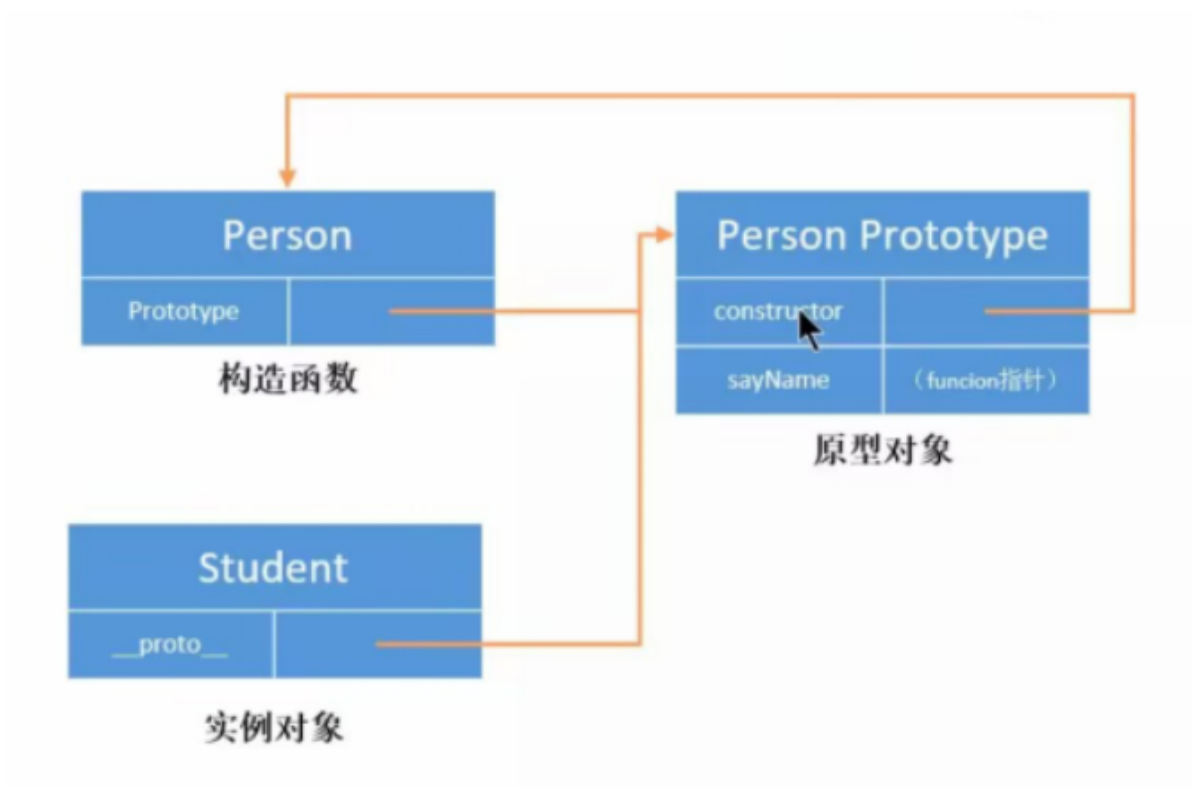
```
Object.prototype.**proto**=null
```

实例对象的 `__proto__` === 构造函数的 `prototype`

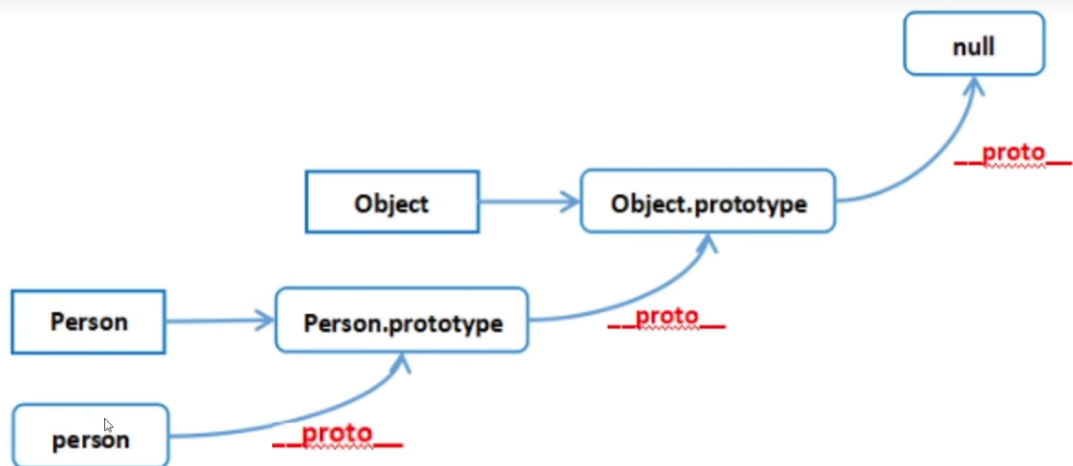
```
const student = new Person();  
student.__proto__===Person.prototype ;//true
```

他们都指向原型对象，原型对象的 `constructor` 指向构造函数，原型对象又会有自己的原型，不断向上查找直到 `Object.prototype.__proto__=null`

```
<html>  
  <head>  
    <meta charset="UTF-8" />  
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />  
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />  
    <title>重写原型对象问题</title>  
  </head>  
  <body>  
    <script>  
      function Person() {}  
      Person.prototype = {  
        constructor: Person,  
        userName: "zhangsan",  
        age: 20,  
        sayHi: function() {  
          console.log(this.userName)  
        }  
      }  
      var person = new Person();  
      person.sayHi(); //zhangsan  
      console.log(Person.prototype.constructor); //Person() { }  
    </script>  
  </body>  
</html>
```

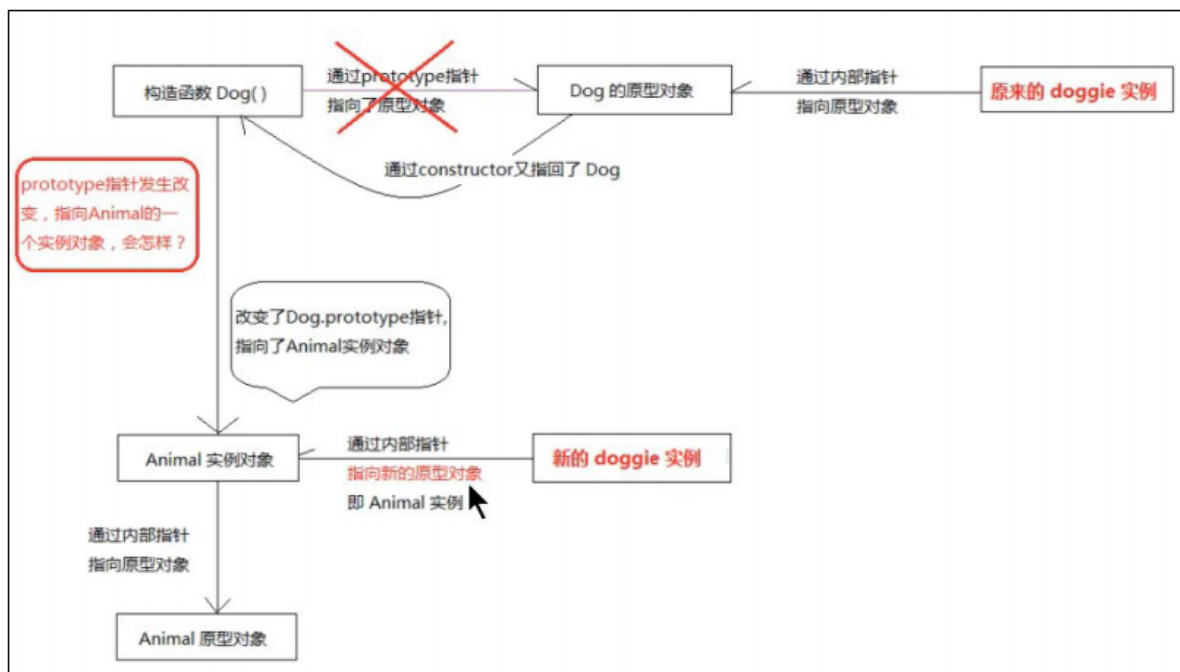


原型链



72、JS实现继承的方法

1) 原型链继承



优点:

1. 实现简单
2. 可以通过子类的实例直接访问父类原型链中的属性或者是方法

```
function Animal(name) {
  this.superType = "Animal";
  this.name = name || "动物";
  this.sleep = function () {
    console.log(this.name + "正在睡觉");
  };
}
Animal.prototype.eat = function (food) {
  console.log(this.name + "正在吃" + food);
};
function Dog(name) {
  this.name = name;
}
//关键
Dog.prototype = new Animal();
Dog.prototype.constructor = Dog;

var doggie = new Dog("lili");
console.log(doggie.superType); //Animal
doggie.sleep(); //lili正在睡觉
doggie.eat("狗粮"); //lili正在吃狗粮
```

缺点:

如果父类某个属性的值是引用类型，某个子类的实例去修改这个属性的值，影响其他子类的值。

```
function Person() {
  this.emotion = ["吃饭", "睡觉"];
}
function Student(id) {
  this.id = id;
}
Student.prototype = new Person();
Student.prototype.constructor = Student;
let stu1 = new Student(10001);
console.log(stu1.emotion); // ["吃饭", "睡觉", "学习"]
stu1.emotion.push("学习");
let stu2 = new Student(10002);
console.log(stu2.emotion); // ["吃饭", "睡觉", "学习"]
```

2) 构造函数继承

```
//构造函数继承
function Person() {
  this.emotion = ["吃饭", "睡觉"];
}
function Student(id) {
  this.id = id;
  //关键
  Person.call(this);
}

let stu1 = new Student(10001);
console.log(stu1.emotion); // [ "吃饭", "睡觉" ]
```

优点:

1. 创建子类，可以向父类构造函数传递参数

```
//构造函数继承
function Person(age) {
  this.emotion = ["吃饭", "睡觉"];
  this.age = age;
}
function Student(id, age) {
  this.id = id;
  //关键
  Person.call(this, age);
}

let stu1 = new Student(10001, 20);
console.log(stu1.emotion);
console.log(stu1.emotion);
console.log(stu2.age);
```

- 2、父类的属性如果是引用类型，子类实例进行修改互不影响

缺点:

- 1、子类只能继承父类构造函数的实例属性和方法，无法继承父类原型对象上的方法

```
//构造函数继承
```

```
function Person() {
  this.emotion = ["吃饭", "睡觉"];
}
Person.prototype.study = function () {
  console.log("好好学习");
};
function Student(id) {
  this.id = id;
  //关键
  Person.call(this);
}

let stu1 = new Student(10001);
console.log(stu1.emotion);
stu1.study(); //报错
```

2、父类中有很多方法，每次创建一个实例都要创建方法，占用内存大

```
//构造函数继承
function Person() {
  this.emotion = ["吃饭", "睡觉"];
  //这里
  this.study = function () {
    console.log("好好学习");
  };
  //
}
function Student(id) {
  this.id = id;
  //关键
  Person.call(this);
}

let stu1 = new Student(10001);
console.log(stu1.emotion);
stu1.study();
```

3) 拷贝继承

优点:

1. 创建子类，可以向父类构造函数传递参数
2. 子类可以继承父类构造函数的实例属性和方法，也可以继承父类原型对象上的方法

缺点:

for in 消耗内存

```
function Person(age) {
  this.emotion = ["吃饭", "睡觉"];
  this.age = age;
  this.study = function () {
    console.log(this.id + "好好学习");
  };
}
Person.prototype.play = function () {
  console.log(this.id + "玩耍" + this.age);
}
```

```

};
function Student(id, age) {
  let person = new Person(age);
  this.id = id;
  for (let key in person) {
    if (person.hasOwnProperty(key)) {
      this[key] = person[key];
    } else {
      Student.prototype[key] = person[key];
    }
  }
}

let stu1 = new Student(10001, 21);
console.log(stu1.emotion);
stu1.study();
stu1.play();

```

4) 组合继承

将原型继承与构造函数继承组合使用

```

//组合继承
function Person(age) {
  this.emotion = ["吃饭", "睡觉"];
  this.age = age;
  this.study = function () {
    console.log(this.id + "好好学习");
  };
}
Person.prototype.play = function () {
  console.log(this.id + "玩耍" + this.age);
};
function Student(id, age) {
  this.id = id;
  //关键
  Person.call(this, age); //第一次
  //关键
}
//关键
Student.prototype = new Person(); //第二次
Student.prototype.constructor = Student;
//关键
let student = new Student(1001, 21);
student.study();
student.play();

```

优点:

1. 子类可以继承父类构造函数的实例属性和方法，也可以继承父类原型对象上的方法
2. 子类可以向父类参数传递

缺点:

1. 父类构造函数被调用两次

父类构造函数中和父类原型中定义相同的方法，子类实例调用时会调用父类构造函数的方法

5) 寄生式组合继承

```
//组合继承
function Person(age) {
  this.emotion = ["吃饭", "睡觉"];
  this.age = age;
  this.study = function () {
    console.log(this.id + "好好学习");
  };
}
Person.prototype.play = function () {
  console.log(this.id + "玩耍" + this.age);
};
function Student(id, age) {
  this.id = id;
  //关键
  Person.call(this, age); //第一次
}

//关键
function Super() {}
Super.prototype = Person.prototype;
Student.prototype = new Super();

Student.prototype.constructor = Student;
let student = new Student(1001, 21);
student.study();
student.play();
```

解决组合继承缺点

73、CSRF (跨站请求伪造攻击)

CSRF 攻击指的是**跨站请求伪造攻击**，攻击者诱导用户进入一个第三方网站，然后该网站向被攻击网站发送跨站请求。如果用户在被攻击网站中保存了登录状态，那么攻击者就可以利用这个登录状态，绕过后台的用户验证，冒充用户向服务器执行一些操作。

CSRF 攻击的**本质是利用 cookie 会在同源请求中携带发送给服务器的特点，以此来实现用户的冒充。**

用户登录了 a.com，有了 cookie，黑客诱导用户访问 b.com，并在其中发起跨站请求 `a.com/api`，服务端收到 cookie 误以为使用户的真实请求就受理了。

常见的 CSRF 攻击有三种：

- GET 类型的 CSRF 攻击，比如在网站中的一个 img 标签里构建一个请求，当用户打开这个网站的时候就会自动发起提交。
- POST 类型的 CSRF 攻击，比如构建一个表单，然后隐藏它，当用户进入页面时，自动提交这个表单。
- 链接类型的 CSRF 攻击，比如在 a 标签的 href 属性里构建一个请求，然后诱导用户去点击。

74、打包防止缓存

浏览器缓存(Caching)可以提升用户的浏览体验，降低服务器压力，但是缓存是不是百利而无一害的呢？

可以思考这样的场景：

用户首次访问下载了index.js到缓存中，在用户第二次访问同一页面之前，我们修改了index.js的源文件并且重新打包上线了index.js，如果此时用户的缓存中还存在之前留下的index.is，那么用户就会加载缓存中的index.js，而不是最新的修改后的index.js文件，如何解决这个问题呢？

修改webpack.prod.js:

```
output:{
  filename: '[name].[contenthash].js'
  path:path.resolve( dirname, '../dist')
}
```

在输出时，为每一个文件名添加contenthash值，当该文件发生改变时，该文件的文件名也会发生改变，这时候浏览器在请求该文件时，就不会命中缓存中过期的文件。

75、webpack 热更新 (HMR)

定义：是一种在运行时更新模块的技术，允许在不刷新整个页面的情况下，替换、添加或删除模块（如 JavaScript、CSS、Vue 组件等）

webpack-dev-server 运行时会创建两个服务器：

1. **静态资源服务器 (Express)**：用于提供打包后的 HTML、JS、CSS 等静态资源，供浏览器访问和解析。
2. **Socket 服务器 (WebSocket)**：用于在服务器和浏览器之间建立长连接，实现双向通信。

热更新流程：

- 当socket server检测到模块发生变化时生成两个文件：
 - **.json (manifest 文件)**：包含新的 hash 值和发生变更的模块信息。webpack 每次编译代码时，都会生成一个新的 hash 值，如果代码发生变更，hash 也会改变。
 - **.js (update chunk 文件)**：存储最新的模块代码。
- 通过 WebSocket，服务器主动将这两个文件推送给浏览器。
- 浏览器接收后，通过 **HMR 运行时** 加载新的模块，并更新页面，而**无需刷新整个页面**。

```
//vue.config.js
const webpack = require('webpack')
module.export = {
  devServer: {
    hot: true
  },
  plugins[
    new webpack.HotModuleReplacementPlugin()
  ]
}
```

76、高阶函数

定义：高阶函数 (Higher-Order Function) 是指接受函数作为参数 或 返回一个函数的函数

常见的高阶函数：promise, setTimeout, arr.map(), arr.sort()

(1) setTimeout

```
setTimeout(() => {  
  console.log("延迟 2 秒执行");  
}, 2000);
```

(2) Promise

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve('数据加载完成');  
  }, 1000);  
});
```

(3) Array.prototype.map()

```
const numbers = [1, 2, 3, 4];  
const squared = numbers.map(num => num * num);  
console.log(squared); // 输出 [1, 4, 9, 16]
```

(4) Array.prototype.sort()

```
const arr = [3, 1, 4, 1, 5, 9];  
arr.sort((a, b) => a - b);  
console.log(arr); // 输出 [1, 1, 3, 4, 5, 9]
```

77、函数柯里化

定义：通过一系列嵌套的函数，每次只接收一个参数，直到所有参数都收集完毕，最后执行原函数。

```
// 普通函数：接收多个参数  
function add(a, b, c) {  
  return a + b + c;  
}  
  
// 柯里化后的函数：每次接收一个参数，返回一个新的函数  
function curriedAdd(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    };  
  };  
}  
  
console.log(curriedAdd(1)(2)(3)); // 输出 6
```

78、react/Vue 中 key 有什么作用？key 的内部原理是什么？为什么遍历列表时，key 最好不要用 index？

虚拟 DOM 中 key 的作用

key 是虚拟 DOM 对象的标识，在更新显示 key 时起着极其重要的作用

当状态中的数据发生变化时，react 会根据新数据生成新的虚拟 DOM，随后 react 进行新虚拟 DOM 与旧虚拟 DOM 的 diff 比较，比较规则如下：

a. 旧虚拟 DOM 找到与新虚拟 DOM 相同的 key：

若虚拟 DOM 中的内容没变，直接使用之前的真实 DOM

若虚拟 DOM 中的内容变了，则生成新的真实 DOM，随后替换掉页面中之前的真实 DOM

b. 旧虚拟 DOM 中未找到与新虚拟 DOM 相同的 key：

根据数据创建新的真实 DOM，随后渲染到页面

react 如果父组件重新渲染，默认情况下子组件也会重新渲染，即使 props 没有改变。

用 index 作为 key 可能会引发的问题

1. 若对数据进行：逆序添加、逆序删除的破坏顺序的操作：

会产生没有必要的真实 DOM 更新===》界面效果没问题，但效率低

2. 如果结构中包含输入类 DOM

会产生错误 DOM 更新===》界面有问题

3. 注意如果不存在对数据的逆序添加、逆序删除的破坏顺序的操作，仅用于渲染列表用于展示，使用 index 作为 key 是没有问题的。

```
<template>
  <ul>
    <li v-for="(item, index) in items" :key="index">
      {{ item.name }}
      <button @click="removeItem(index)">删除</button>
    </li>
  </ul>
</template>

<script>
export default {
  data() {
    return {
      items: [
        { name: "苹果" },
        { name: "香蕉" },
        { name: "橙子" }
      ]
    };
  },
  methods: {
    removeItem(index) {
      this.items.splice(index, 1);
    }
  }
};
</script>
```

```
//删除第 1 个元素（苹果）时，其他 index 变化
items = [
  { name: "香蕉" }, // 现在 index = 0
  { name: "橙子" }  // 现在 index = 1
];
//Vue 发现 key=0 仍然存在，于是错误地复用原来的 li 组件，而不是重新创建
//香蕉 组件 复用了 苹果 组件，导致 UI 不更新或数据错乱。
```

开发中如何选择 key?

最好使用每条数据唯一标识作为 key，比如 id、手机号、身份证号、学号等唯一值。

如果确定只是简单的展示数据，用 index 也是可以的。

79、和后端对接接口，通常采用什么样的形式？

前后端之间使用 json 进行交流，主要用 restful 规范

重点：定义接口规范，规范内容一般包含**请求路径**、**请求方法**(get/post/put/delete/update)、**请求参数**和**响应参数**

一个网址就代表一种资源，资源作为网址，只能有名词(复数)，不能有动词，并且所有名词要与数据库表名对应

因为后台数据很多时，使用不可能将所有全部返回给用户，所以要过滤信息，指定返回记录的数量

```
fetch('https://api.example.com/data?limit=10')
  .then(response => response.json())
  .then(data => console.log(data));
```

服务器要向前端返回状态码和信息，告诉我们能够成功获取响应数据时的状态为多少

前后端数据名与数据类型要统一，避免无法获取到正确数据的情况。因为 json 传递的是字符串，前端获取后要进行对应的类型转换。避免后端 int 123-->json "123" 前端解析成"123"的情况

80、移动端适配

在 html 的 head 中添加

```
<meta
  name="viewport"
  content="width=device-width, initial-scale=1.0,
  user-scalable=no, minimum-scale=1.0, maximum-scale=1.0"
/>
```

统一标准**视口**：

- 设置内容宽度等于设备宽度 `width=device-width`
- 初始缩放、最大缩放为 1: `initial-scale=1.0, maximum-scale=1.0`
- 不允许用户缩放 `user-scalable=no`

rem 是一个倍数单位，他是基于 HTML 的 font-size 的倍数

```
<!DOCTYPE html>
<html lang="en">
  <head>
```

```
<meta charset="UTF-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<meta
  name="viewport"
  content="width=device-width, initial-scale=1.0, user-scalable=no, minimum-
scale=1.0, maximum-scale=1.0"
/>
<title>Document</title>
<style>
  .div {
    width: 5rem;
    height: 5rem;
    background-color: pink;
  }
</style>
</head>

<body>
  <div class="div"></div>
  <script>
    document.documentElement.style.fontSize = `${screen.width / 10}px`;
    window.addEventListener("resize", () => {
      document.documentElement.style.fontSize = `${screen.width / 10}px`;
    });
  </script>
</body>
</html>
```

81、JavaScript 事件循环 (Event Loop)

JavaScript 是 **单线程** 运行的，为了避免长时间执行的任务阻塞代码运行，它采用了一种 **事件循环 (Event Loop)** 机制，将代码分为 **同步任务** 和 **异步任务**。

执行机制

1. **同步任务**：直接在 **主线程 (执行栈)** 上执行。
2. **异步任务**：由 浏览器或 Node.js 的宿主环境 处理，如：
 - 定时器 (`setTimeout`、`setInterval`)
 - DOM 事件 (`click`、`mousemove` 等)
 - 网络请求 (`fetch`、`XMLHttpRequest`)
 - Promise、`async/await` 等

事件循环流程

1. **同步任务** 直接执行，放入 **执行栈 (Call Stack)** 依次运行。
2. **异步任务** 先交给 **宿主环境** 处理，并加入相应的 **任务队列 (Task Queue)**。
3. 主线程执行完所有同步任务后，检查任务队列中是否有任务：
 - **宏任务 (Macro Task) 队列**：如 `setTimeout`、`setInterval`、I/O 操作、`requestAnimationFrame`。
 - **微任务 (Micro Task) 队列**：如 `Promise.then`、`MutationObserver`、`queueMicrotask`。
4. **优先执行微任务队列** 中的所有任务，再执行下一个宏任务。
5. 重复上述过程，这就是 **事件循环 (Event Loop)**。

简化理解

1. **同步代码** 先执行。
2. **异步代码** 交给宿主环境，待时机成熟后放入 **任务队列**。
3. **同步任务执行完毕** 后，依次执行 **微任务队列**，再执行 **宏任务队列**。
4. **不断循环检查** 任务队列，确保 JavaScript 能够持续运行，处理异步任务。

这样，JavaScript **即使是单线程** 也能高效处理异步操作，提高性能和用户体验。

82、为什么非对称加密比对称加密慢

1. 计算复杂度不同

- 对称加密 主要依赖 位运算（如 异或 XOR、移位运算 等），计算简单，速度快，尤其在 硬件加速 的支持下，性能更优。
- 非对称加密（如 RSA）涉及 大数运算（如 大数乘法、模幂运算），计算量大，速度较慢。

2. 密钥长度不同

- 对称加密 通常使用 128/256 位 的密钥，计算量相对较小。
- 非对称加密 由于安全性要求更高，通常使用 1024/2048/4096 位 的密钥，需要更多计算资源。

3. 适用场景不同

- 对称加密 适用于 大数据量的加密（如文件加密、磁盘加密）。
- 非对称加密 主要用于 密钥交换、数字签名，不会直接加密大量数据。

总结：

对称加密速度快，因其算法简单，主要依赖位运算。

非对称加密速度慢，因其涉及复杂的大数运算，如 大数乘法、模幂运算。

83、nginx 超时时间配置

send_timeout time：设置发送响应到客户端的超时时间，默认为 60s，此超时时间指的也是**两次成功写操作间隔时间**，而不是发送整个响应的超时时间。如果在此超时时间内客户端没有接收任何响应，则 Nginx 关闭此连接。

Nginx 采用 **两次写操作间隔超时**，而不是 **整个响应的超时**，主要是因为：

1. **HTTP 响应是分块发送的**，不能固定总超时时间。
2. **支持大文件传输**，避免传输未完成就被强制关闭。
3. **防止无效连接长期占用资源**，提升服务器性能。
4. **允许流式传输**（如 WebSocket、Chunked 编码）。

🔗 这样，Nginx 在**高并发和大数据传输**场景下能保持更好的稳定性和资源管理。

84、typeof 和 object.prototype.toString.call()

数据类型	typeof 结果	Object.prototype.toString.call() 结果	转换为字符串的结果	转换为数字值的结果
123	"number"	"[object Number]"	"123"	123
"hello"	"string"	"[object String]"	"hello"	12("12"),0(""),NaN("ab1c")
true	"boolean"	"[object Boolean]"	"true"/"false"	1(true)/0(false)
null	"object" (错误)	"[object Null]" (正确)	"null"	<u>0</u>
undefined	"undefined"	"[object Undefined]"	"undefined"	NaN
{ } (对象)	"object"	"[object Object]"	"[object object]"	不能转换, 报错
[] (数组)	"object"	"[object Array]"	"1,2,3,你好"	1([1])/NaN([1,2,3])
function	"function"	"[object Function]"		
new Date()	"object"	"[object Date]"		
/abc/ (正则)	"object"	"[object RegExp]"		

11. 其他值到字符串的转换规则？

- Null 和 Undefined 类型，null 转换为 "null"，undefined 转换为 "undefined"，
- Boolean 类型，true 转换为 "true"，false 转换为 "false"。
- Number 类型的值直接转换，不过那些极小和极大的数字会使用指数形式。
- Symbol 类型的值直接转换，但是只允许显式强制类型转换，使用隐式强制类型转换会产生错误。
- 对普通对象来说，除非自行定义 toString() 方法，否则会调用 toString() (Object.prototype.toString()) 来返回内部属性 [[Class]] 的值，如 "[object Object]"。如果对象有自己的 toString() 方法，字符串化时就会调用该方法并使用其返回值。

12. 其他值到数字值的转换规则？

- Undefined 类型的值转换为 NaN。
- Null 类型的值转换为 0。
- Boolean 类型的值，true 转换为 1，false 转换为 0。
- String 类型的值转换如同使用 Number() 函数进行转换，如果包含非数字值则转换为 NaN，空字符串为 0。
- Symbol 类型的值不能转换为数字，会报错。
- 对象（包括数组）会首先被转换为相应的基本类型值，如果返回的是非数字的基本类型值，则再遵循以上规则将其强制转换为数字。

为了将值转换为相应的基本类型值，抽象操作 ToPrimitive 会首先（通过内部操作 DefaultValue）检查该值是否有 valueOf() 方法。如果有并且返回基本类型值，就使用该值进行强制类型转换。如果没有就使用 toString() 的返回值（如果存在）来进行强制类型转换。

如果 valueOf() 和 toString() 均不返回基本类型值，会产生 TypeError 错误。